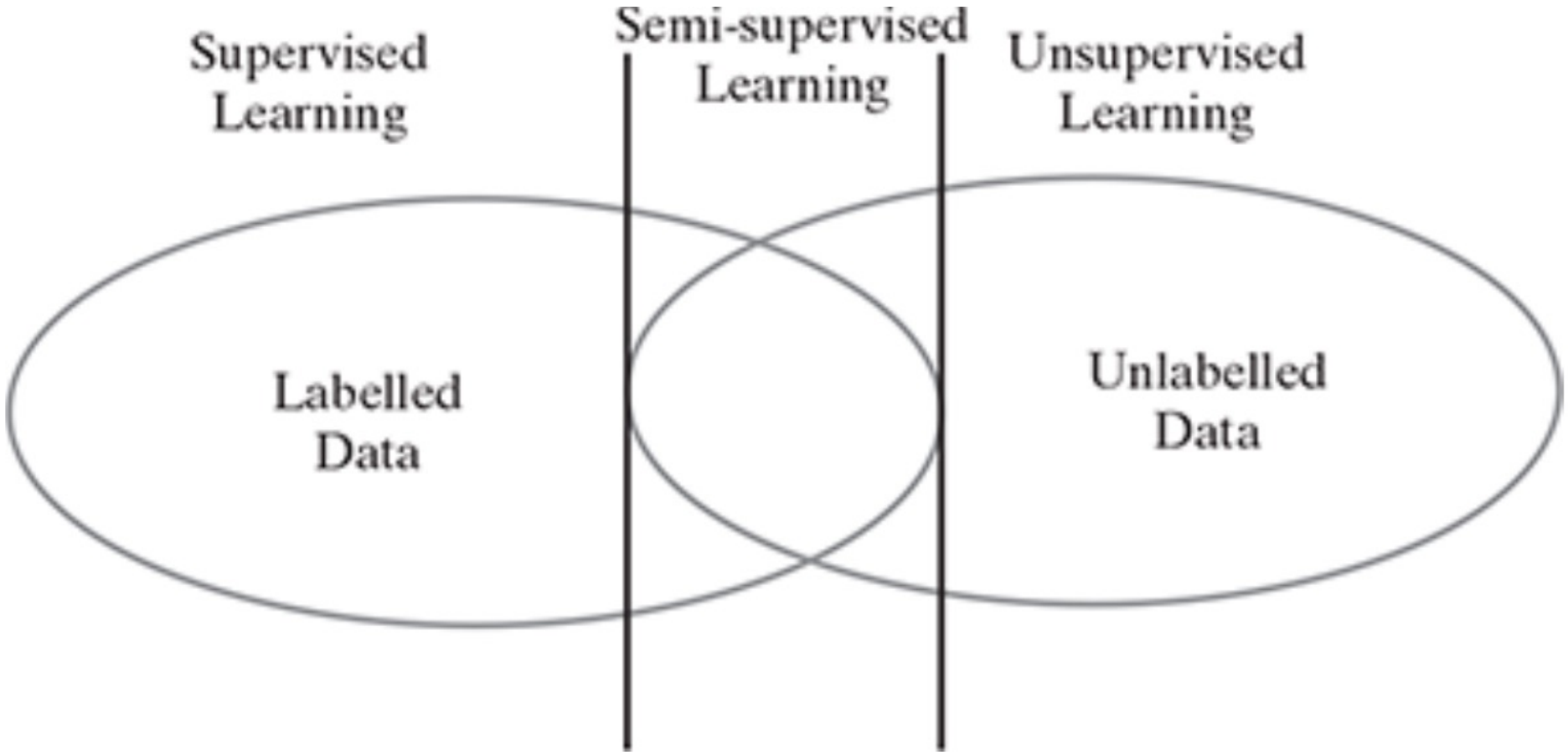
Supervised Learning

Supervised Learning

- In supervised learning, the labelled training data provide the basis for learning.
- According to the definition of machine learning, this labelled training data is the experience or prior knowledge or belief.
- It is called supervised learning because the process of learning from the training data by a machine can be related to a teacher supervising the learning process of a student who is new to the subject.
- Here, the teacher is the training data.
- Training data is the past information with known value of class field or ‘label’.
- Hence, we say that the ‘training data is labelled’ in the case of supervised learning .
- Contrary to this, there is no labelled training data for unsupervised learning.
- Semi-supervised learning, uses a small amount of labelled data along with unlabelled data for training.

Supervised Learning



Supervised Learning

- In supervised learning, models are trained using labeled data to predict outcomes for new cases.
- In the hospital scenario, past patient records are used to train a classification model to identify patients in general wards who may need ICU care. The model predicts whether a newly admitted patient is high-risk (likely to deteriorate) or low-risk based on their test results.
- Other examples of supervised learning include:
  - Predicting game outcomes from past results
  - Classifying tumors as malignant or benign based on medical data
  - Forecasting prices in real estate, stocks, and similar domains

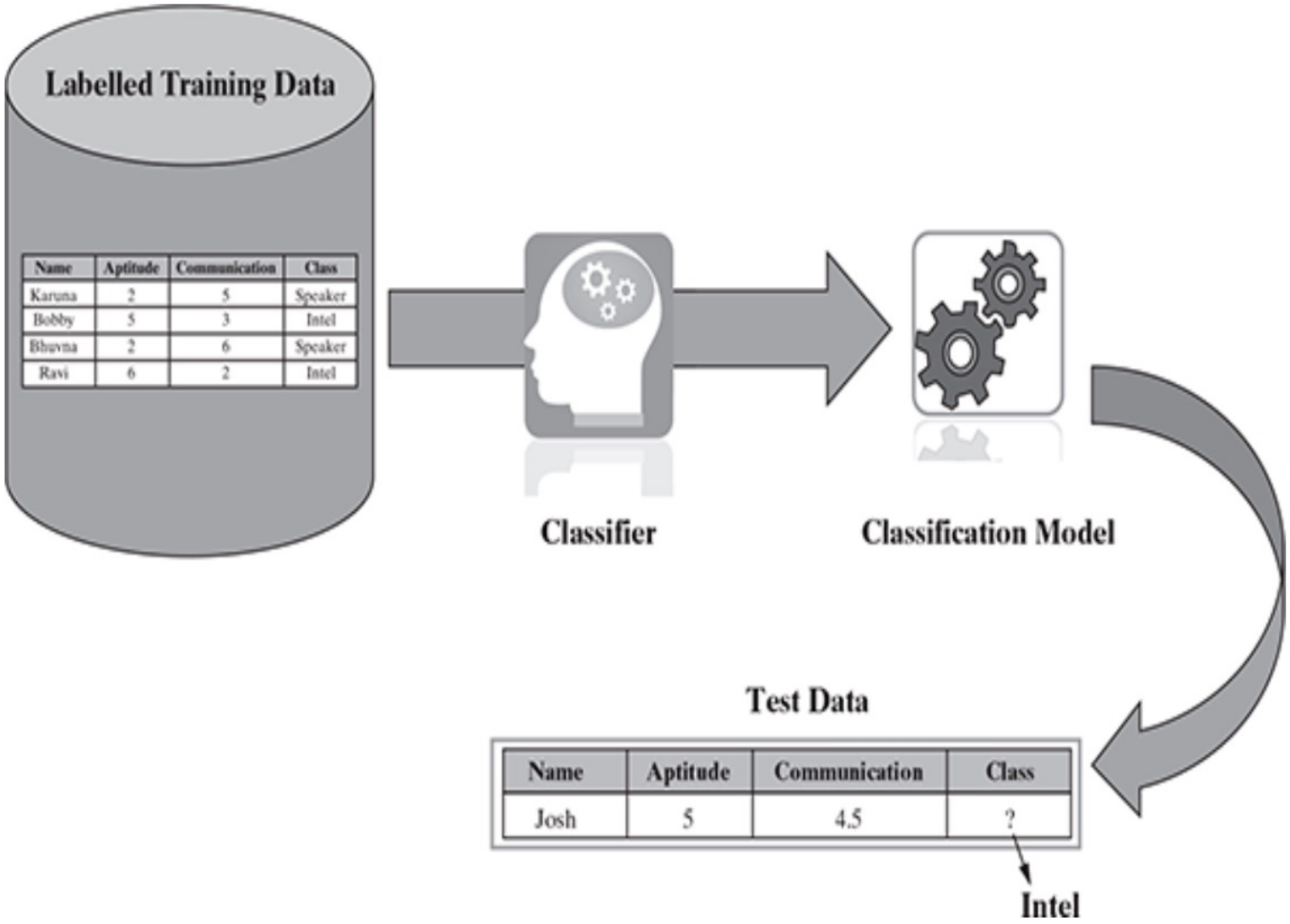
Classification Model

- Classification is a type of supervised learning used to predict categorical variables (like 'red' or 'benign'), while regression predicts numerical variables (like price or weight).
- In classification, labeled training data helps a model learn to assign a class label to new test data. The quality of predictions depends heavily on the quality of the training data.
- An example of classification is fraud detection in banking — models trained on past transactions (fraudulent and non-fraudulent) predict whether new ones are suspicious.

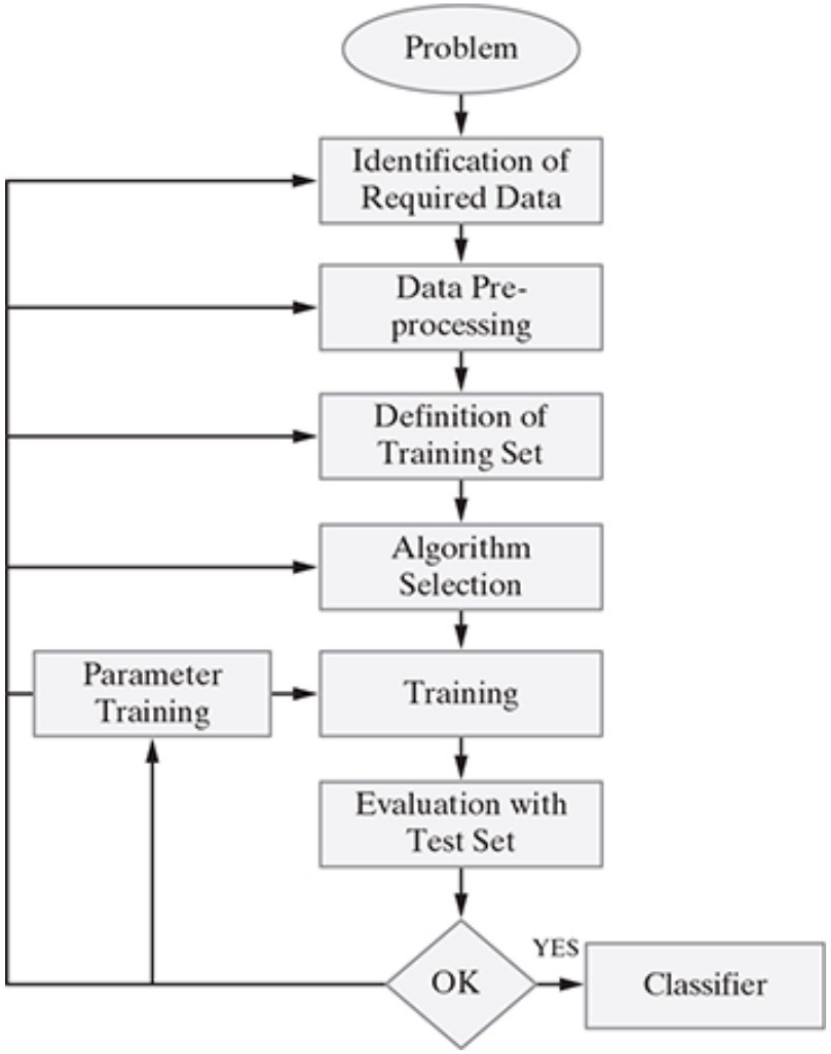
Classification Model

- Common examples of classification problems include:
  - Image classification
  - Disease prediction
  - Win–loss prediction for games
  - Natural calamity prediction
  - Handwriting recognition
- In short, classification aims to assign a category or class to new data based on patterns learned from labeled examples.

Classification Model



Classification Learning Steps



1. **Problem Identification**: Clearly define a well-structured problem with measurable goals and long-term benefits.
2. **Data Identification**: Determine the exact dataset required for the problem. For example, predicting tumor malignancy needs relevant patient data.
3. **Data Pre-processing**: Clean and transform raw data by removing irrelevant elements, handling missing values, and ensuring the dataset is analysis-ready.
4. **Training Data Definition**: Decide what the training examples will represent (e.g., handwriting analysis might use single letters or sentences). Collect input data (X) and corresponding outputs (Y) that reflect real-world scenarios.
5. **Algorithm Selection**: Choose a suitable learning algorithm (such as decision trees, SVM, or logistic regression) based on problem type and data characteristics.
6. **Training**: Use the chosen algorithm to learn patterns from the training dataset. Adjust parameters as needed to improve performance, sometimes using a validation subset of data.
7. **Evaluation**: Test the trained model on a separate test dataset to measure accuracy. If performance is inadequate, retraining or fine-tuning may be necessary.

Common Classification Models

- Let us now delve into some common classification algorithms.
- Following are the most common classification algorithms.
  1. k-Nearest Neighbour (kNN)
  2. Decision tree
  3. Random forest
  4. Support Vector Machine (SVM)

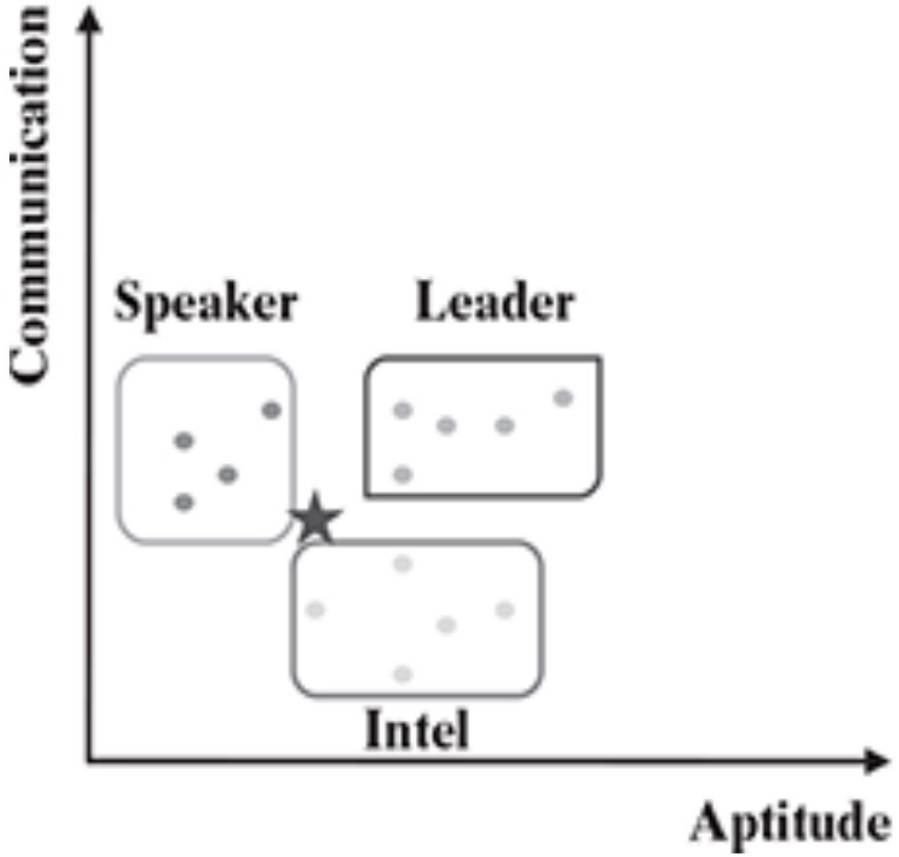
k -Nearest Neighbour (kNN)

- The k-Nearest Neighbour (kNN) algorithm is a simple but powerful classification method based on the idea that similar data points exist close to each other in feature space.
- The name originates from the observation that people with similar characteristics tend to live near one another.
- In kNN, an unlabelled data point's class is determined by examining the classes of its nearest neighbours in the training dataset.
- The algorithm works well for tasks where similarity measures are meaningful.
- It doesn't assume any prior distribution of data and uses the existing labelled data for predictions.
- In essence, instead of learning an explicit model, kNN makes predictions directly using stored data points, relying on their proximity to the target instance to assign or predict a label based on majority voting among the nearest neighbours.

How kNN Works?

- To understand kNN, consider an example of a student dataset with features like *Aptitude* and *Communication skills*, and classes labeled as *Leader*, *Speaker*, or *Intel*.
- The goal is to classify a new student (test data) using existing known data points (training data).
- kNN calculates the similarity between the test instance and all training instances, typically using Euclidean distance, which measures straight-line distance between two points in multi-dimensional space.
- The test instance is then classified by the majority class among its *k* closest neighbours.
- For example, if *k* = *3*, the algorithm picks three nearest data points; if two of these belong to class ‘Intel’ and one to ‘Leader,’ the test point is classified as ‘Intel’. Selecting an appropriate *k* is crucial for balancing noise sensitivity and generalization.

How kNN Works?



Training Data

Test Data

Name	Aptitude	Communication	Class
Karuna	2	5	Speaker
Bhuvna	2	6	Speaker
Gaurav	7	6	Leader
Parul	7	2.5	Intel
Dinesh	8	6	Leader
Jani	4	7	Speaker
Bobby	5	3	Intel
Parimal	3	5.5	Speaker
Govind	8	3	Intel
Susant	6	5.5	Leader
Gouri	6	4	Intel
Bharat	6	7	Leader
Ravi	6	2	Intel
Pradeep	9	7	Leader
★ Josh	5	4.5	???

How kNN Works?

- k-Nearest Neighbors (kNN) can classify new data by comparing it to previously labeled examples. In the student data scenario, the goal is to determine which class label (such as 'Intel' or 'Leader') should be assigned to student Josh based on his measured attributes.
- The algorithm works by computing distances between Josh and all students in the training set, using measures like Euclidean distance.
- When k=1, only the single nearest neighbour is considered. Here, Gouri is Josh’s closest neighbour (distance 1.118), so Josh is directly assigned Gouri’s class label: ‘Intel’.
- When k=3, the algorithm considers Josh's three nearest neighbours: Gouri (1.118), Susant (1.414), and Bobby (1.5). Both Gouri and Bobby are labeled 'Intel', and Susant is labeled 'Leader’.
- The class is then determined by a majority vote: with two out of three neighbours labeled 'Intel', Josh is classified as 'Intel’.
- This approach generalizes for any value of k*k*: the most common class among the k*k* nearest neighbours sets the predicted label.
- This illustrates how kNN uses proximity and majority voting to predict outcomes for new cases.

How kNN Works?

Name	Aptitude	Communication	Class	Distance	$k = 1$	$k = 2$	$k = 3$
Karuna	2	5	Speaker	3.041			
Bhuvna	2	6	Speaker	3.354			
Parimal	3	5.5	Speaker	2.236			
Jani	4	7	Speaker	2.693			
Bobby	5	3	Intel	1.500			1.500
Ravi	6	2	Intel	2.693			
Gouri	6	4	Intel	1.118	1.118	1.118	1.118
Parul	7	2.5	Intel	2.828			
Govind	8	3	Intel	3.354			
Susant	6	5.5	Leader	1.414			
Bharat	6	7	Leader	2.693			
Gaurav	7	6	Leader	2.500			
Dinesh	8	6	Leader	3.354			
Pradeep	9	7	Leader	4.717			
Josh	5	4.5	???				

Distance Calculation and Majority Voting

- The Euclidean distance formula measures the closeness between data points based on their feature values.
- For two-feature data like Aptitude (f1) and Communication (f2), the distance between data elements d_1 and d_2 is computed as
- Distances are calculated between the test record and all training records. Once distances are found, the algorithm ranks them to find the *k* smallest ones—these represent the nearest neighbours.
- The test data's predicted class is the most frequent among these *k* neighbours (majority rule).
- Setting *k = 1* uses only the nearest neighbour, while larger *k* values average across more samples.
- Extreme *k* values are undesirable—large *k* can oversmooth and small *k* is prone to noise.

$$\text{Euclidean distance} = \sqrt{(f_{11} - f_{12})^2 + (f_{21} - f_{22})^2}$$

where f_{11} = value of feature f_1 for data element d_1

f_{12} = value of feature f_1 for data element d_2

f_{21} = value of feature f_2 for data element d_1

f_{22} = value of feature f_2 for data element d_2

Choosing the Value of k

- Choosing the optimal *k* is essential for achieving accurate classification.
- If *k* is too large, the algorithm oversimplifies by assigning the dominant class label to most points, ignoring fine distinctions between classes.
- If *k* is too small, such as 1, the algorithm becomes overly sensitive to outliers or noise in the data.
- Common strategies for selecting *k* include setting *k* to the square root of the number of training samples or experimenting with multiple *k* values using a validation set to determine the best performance.
- Some implementations employ weighted voting, giving more influence to closer neighbours while diminishing the impact of farther ones.
- This improves performance when data points in the neighbourhood vary in distance from the test sample.

Step-by-Step kNN Algorithm

kNN operates through a sequence of clear steps:

1. Input: A labelled training dataset, one or more test points, and a chosen *k* value.
2. Distance Calculation: For each test point, compute distances (commonly Euclidean) to all training points.
3. Neighbour Selection: Identify *k* training samples with the smallest distances.
4. Class Assignment:
   - If *k = 1*, assign the nearest neighbour's class.
   - For *k > 1*, assign the most frequent class (majority voting).
     This process repeats for every test element in the dataset. Because no explicit training or model building occurs, classification happens at runtime directly based on the training examples.

Lazy Learning Nature

- kNN is known as a lazy learner because it performs virtually no model training.
- Unlike algorithms that generalize from data to create a mathematical model (eager learners like decision trees or logistic regression), kNN simply stores the training dataset.
- It defers computation until a new test instance needs to be classified, at which point it calculates distances and decides a label.
- This makes the training phase extremely fast (since it only involves storing the data) but the prediction phase potentially slow, as all calculations are done at runtime.
- This behaviour means kNN excels in small to moderately sized datasets but can struggle with scalability on very large datasets.

Strengths, Weaknesses, and Applications

Strengths:

- Simple and intuitive—easy to understand and implement.
- Very effective for recommender systems (suggesting similar items based on user preferences).
- Minimal training time since it stores rather than models data.

Weaknesses:

- High computational cost during prediction because of distance computations.
- Large memory requirement to store entire training datasets.
- Performance degrades when data has many irrelevant or noisy features.

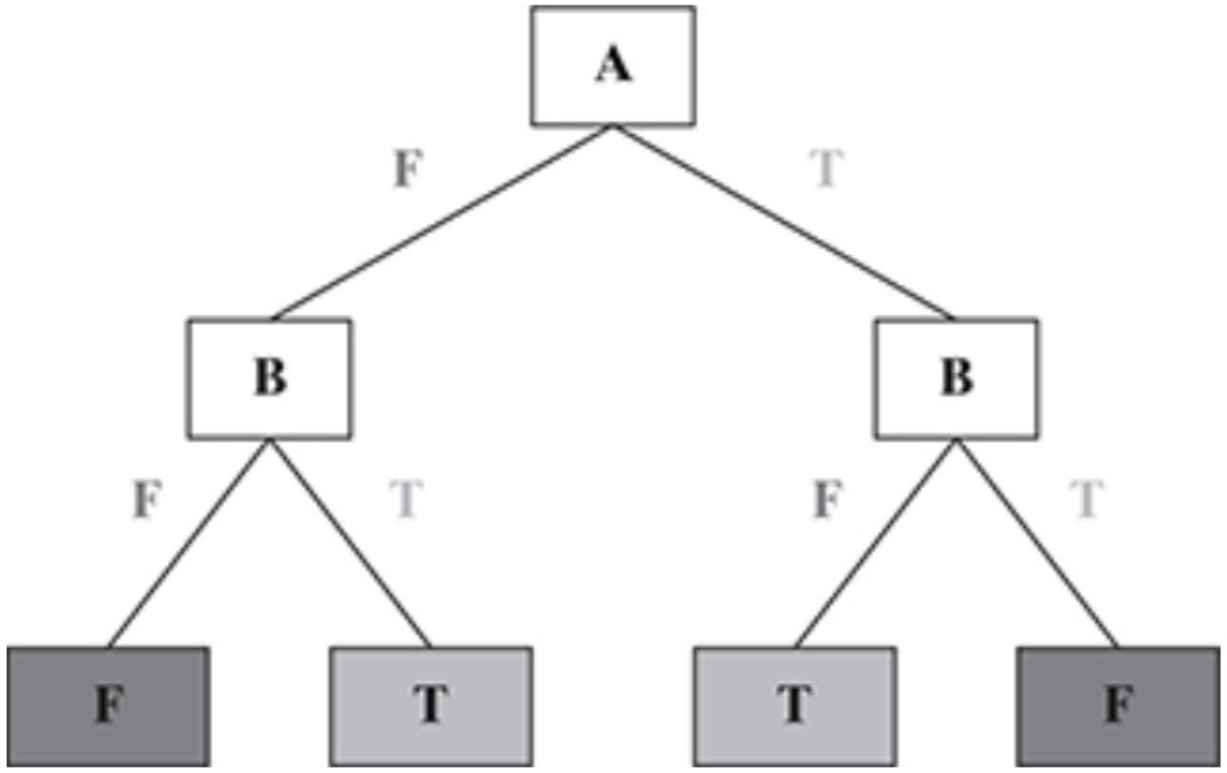
Applications:

- Recommender systems (product suggestions).
- Information retrieval—finding documents similar to a given one (concept search).
- Pattern recognition and anomaly detection.

Decision Trees

- Decision trees are popular supervised learning algorithms used for classification and regression.
- They create a model resembling a tree structure where each internal node represents a test on an attribute, each branch corresponds to an outcome of the test, and each leaf node holds a class label or value.
- The main objective is to partition the data such that each resulting subset ideally contains instances of a single class, making the subsets "pure."
- Decision tree algorithms include ID3, C4.5, CART, and CHAID, differing mainly in how they select attributes for splits.
- The strongest attribute is the one that best separates the data, measured using criteria like Entropy, Information Gain, Gini Index, or Chi-square.
- The tree recursively splits data until stopping criteria are met, producing interpretable "if-then" rules and enabling accurate predictions on new data.

Decision Trees



Each internal node (represented by boxes) tests an attribute (represented as ‘A’/‘B’ within the boxes).

Each branch corresponds to an attribute value (T/F) in the above case.
Each leaf node assigns a classification.

The first node is called as ‘Root’ Node.

Branches from the root node are called as ‘Leaf’ Nodes where ‘A’ is the Root Node (first node). ‘B’ is the Branch Node.

‘T’ & ‘F’ are Leaf Nodes.

Thus, a decision tree consists of three types of nodes:

- Root Node
- Branch Node
- Leaf Node

Creating a Decision Tree

- Decision trees are built by recursively partitioning the dataset based on features that best predict the target outcome.
- The process starts with the entire dataset at the root node.
- The algorithm selects the feature that maximizes homogeneity of partitions toward the target class, generally by measuring information gain or reduction in impurity (such as entropy or Gini index).
- Each split divides the data into subsets where examples largely belong to a single class, creating pure partitions.
- Partitioning continues recursively on each subset until stopping criteria like all samples belonging to the same class, exhaustion of features, or pre-defined tree depth are met.
- The result is a tree structure with root, branch, and leaf nodes, where leaves represent predicted classes.
- This method ensures no loss of information if the splits perfectly separate classes, allowing the tree to model the training data accurately.

Searching a Decision Tree

- Searching the decision tree to make predictions involves traversing from the root node based on the attributes of the input instance down to a leaf node representing the predicted class.
- Two main searching methods are exhaustive search and branch-and-bound.
- Exhaustive search explores all possible paths to classify an input but can be inefficient for large trees.
- Branch-and-bound leverages heuristics and the best known solution to prune the search space, skipping branches unlikely to improve prediction.
- This makes prediction faster without losing information.
- By following the path dictated by attribute values, searching a decision tree is intuitive and interpretable, allowing identification of the reasoning behind predictions.
- Efficient search combined with careful tree construction guarantees accurate classification with no information loss.

Entropy and Information Gain

- Entropy measures the impurity or disorder in a dataset.
- Defined mathematically, entropy is higher when classes are mixed evenly and zero when all instances belong to one class.
- Information Gain is the key metric for attribute selection, computed as the reduction in entropy after a dataset is split on an attribute.
- The attribute yielding the highest Information Gain is chosen for splitting.
- For example, in a dataset predicting job offers, entropy is calculated before and after splitting by attributes like CGPA or Communication skills, comparing the resulting values.
- The attribute that produces the largest decrease in entropy (or highest gain) creates the most homogeneous subsets, improving classification.
- This recursive process continues, splitting each branch similarly.

Entropy and Information Gain

Entropy (S):

Entropy measures the impurity or uncertainty in a dataset S containing c classes. It is defined as:

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2 p_i$$

where p_i is the proportion of instances in class i in dataset S.
The entropy is zero when all instances belong to a single class and is maximum when the classes are uniformly distributed.

Refer to Section 7.5.2.3 Entropy of a decision tree

Entropy and Information Gain

The information gain for a particular feature A is calculated as the difference between the entropy before the split and the entropy after the split:

$$\text{Information Gain}(S, A) = \text{Entropy}(S_{\text{before split}}) - \text{Entropy}(S_{\text{after split}})$$

where:

- $S_{\text{before split}}$ is the dataset before split,
- $S_{\text{after split}}$ is the dataset after splitting based on attribute A.

This formula quantifies how much the uncertainty (entropy) is reduced by splitting the data on feature A.

Entropy and Information Gain

- For calculating the entropy after a split, the entropy must be computed for all the resulting partitions.
- Then, the total entropy after the split is obtained by taking a weighted summation of the entropies of each partition.
- The weights used are the proportions of examples that fall into each partition relative to the whole dataset.
- This means each partition's entropy contributes proportionally to the overall entropy after the split, reflecting both the size and impurity of the partitions.
- This weighted entropy after the split is used in the computation of Information Gain, which measures the effectiveness of the split in reducing uncertainty in the data.

Note : Refer to Section 7.5.2.4 How Information Gain is Calculated for an example

Recursive Tree Construction

The decision tree is grown top-down, recursively partitioning data:

1. Calculate entropy for the current dataset.
2. For each attribute, compute Information Gain by splitting the dataset based on attribute values.
3. Choose the attribute with the highest Information Gain as the decision node.
4. Partition the dataset accordingly.
5. Repeat these steps for each subset until all subsets are pure or meet stopping criteria. This results in a tree where decisions about class membership follow a path determined by feature values. The process ensures that each split progressively reduces class impurity, leading to focused, interpretable classification rules reflecting the data's structure.

Pruning and Overfitting

- Decision trees can easily overfit training data by growing excessively large, capturing noise and specifics rather than general patterns. To prevent this, pruning techniques are applied:
  - Pre-pruning stops growth early based on criteria such as a minimal number of instances or maximum tree depth.
  - Post-pruning allows a full tree to form then removes branches that do not improve performance, simplifying the tree.
    Pruning enhances generalization by reducing complexity and overfitting, making the tree better suited for unseen data. There is a trade-off between capturing enough complexity and avoiding noise, requiring careful tuning of pruning parameters for best results.

Strengths and Weaknesses

Strengths: Decision trees generate easily interpretable models with straightforward decision rules, suitable for both numerical and categorical data.

They work well with small to large datasets and provide insights into feature importance. They handle missing values and noisy data robustly.

Weaknesses: They can be biased towards features with many levels, prone to overfitting or underfitting without proper pruning, and may be computationally expensive for many classes or large data.

Large trees become complex and hard to interpret.

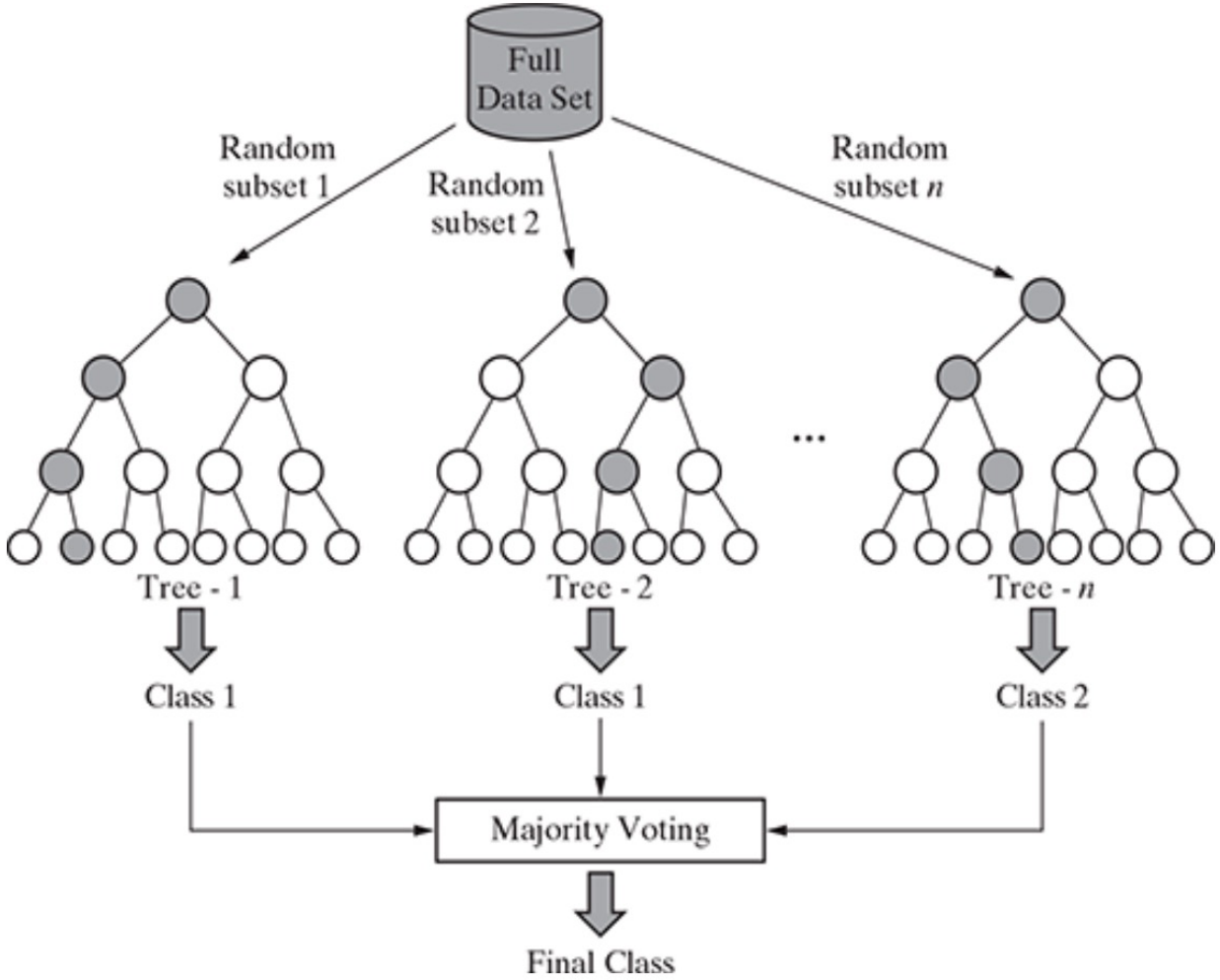
Applications of Decision Trees

- Decision trees excel in situations with finite, distinct feature sets, such as classification of students based on scores or credit risk assessment.
- They can handle both binary and multi-class problems and can be extended to continuous attributes.
- They are commonly used in finance for loan approval, healthcare for disease diagnosis, marketing for customer segmentation, and many other domains requiring transparent, rule-based decision-making.
- Their robustness to noisy data and missing attributes further broadens their usability, making decision trees a versatile tool for supervised learning tasks.

Random Forests

- Random forest is an ensemble machine learning model composed of multiple decision tree classifiers.
- It combines their outputs to form a more accurate and stable prediction.
- The trees are built using bagging (bootstrap aggregating), where each tree is trained on a random subset of data and features.
- This randomness ensures diversity among trees and reduces overfitting.
- The final prediction is made through majority voting (for classification) or averaging (for regression).
- Random forests outperform single decision trees by enhancing generalization and minimizing variance.
- They are robust against noisy and missing data, maintaining accuracy even in complex, high-dimensional problems.
- The model is particularly effective when individual trees are weak learners but collectively provide a strong predictive ensemble.

Random Forests



After the random forest is generated by combining the trees, majority vote is applied to combine the output of the different trees.

The result from the ensemble model is usually better than that from the individual decision tree models.

Working Mechanism and Steps

- The algorithm operates by first randomly selecting subsets of features and samples from the training dataset.
- For each subset, a decision tree is built using the best split logic. The process repeats for multiple trees, creating a large ensemble.
- Each tree votes independently, and the majority vote determines the final output.
- Random forests also use out-of-bag (OOB) samples—data not included in the bootstrap sample—to estimate model accuracy without needing a separate validation set.
- This gives a nearly unbiased error estimate.
- While too many trees may cause computational overhead, they rarely lead to severe overfitting thanks to the averaging effect.
- Thus, the model balances low bias and low variance effectively.

How does random forest work

Building the Forest

1. Given a dataset with *N* features, randomly select a subset of *m* features (*m*<*N*) for each tree.
2. Randomly pick data instances (observations) for each tree using sampling with replacement (bootstrapping).
3. For the selected m*m* features, use the best split principle to determine how to split nodes and grow the tree.
4. Continue splitting nodes until the tree reaches its maximum possible size, without pruning.

How does random forest work

Making Predictions

5. Repeat steps 1–4 to build and train *n* decision trees, each with different data and feature subsets.
6. For prediction, each tree votes independently on the class assignment.
7. The final class is assigned based on the majority vote from all *n* trees, ensuring robust and accurate results

What is Out-of-Bag (OOB) Error?

- Out-of-bag (OOB) error is an internal validation method used in random forests to estimate prediction error without needing a separate test set.
- When building each decision tree, the algorithm uses a bootstrap sample—randomly selecting data points with replacement from the training set.
- As a result, about 63% of the data is used to train each tree, while the remaining 37% (the "out-of-bag" samples) are left out for that tree.
- For every data point, predictions are made using only the trees where that point was not included in the training sample.
- The OOB error is then calculated as the average error across all these predictions.
- This approach provides an unbiased estimate of model performance, similar to cross-validation, but is more efficient since it leverages the training process itself

Advantages and Limitations of OOB Error

Advantages: OOB error offers a reliable, nearly unbiased estimate of a random forest's generalization error without the need for a separate validation set or cross-validation, saving both time and computational resources.

It allows the entire dataset to be used for both training and validation, which is especially valuable for small datasets.

OOB error can also help in tuning hyperparameters, such as the number of trees or features considered at each split.

Limitations: OOB error may be less accurate for very small datasets or highly imbalanced classes, and its reliability depends on the randomness of the bootstrap process.

In some cases, especially with certain parameter settings or data types, OOB error can slightly overestimate the true prediction error.

Despite these caveats, OOB error remains a practical and widely used tool for model evaluation in random forests

Strengths, Weaknesses, and Applications

- Random forest is a highly efficient and versatile machine learning algorithm with several key strengths.
- It can process large and complex datasets quickly, making it suitable for big data applications.
- The algorithm is robust in handling missing data, maintaining high precision even when a significant portion of the data is absent.
- Random forest also excels at balancing errors in datasets with imbalanced class populations, ensuring fair and accurate predictions. It provides valuable insights into feature importance, helping users identify which variables most influence the model's decisions.

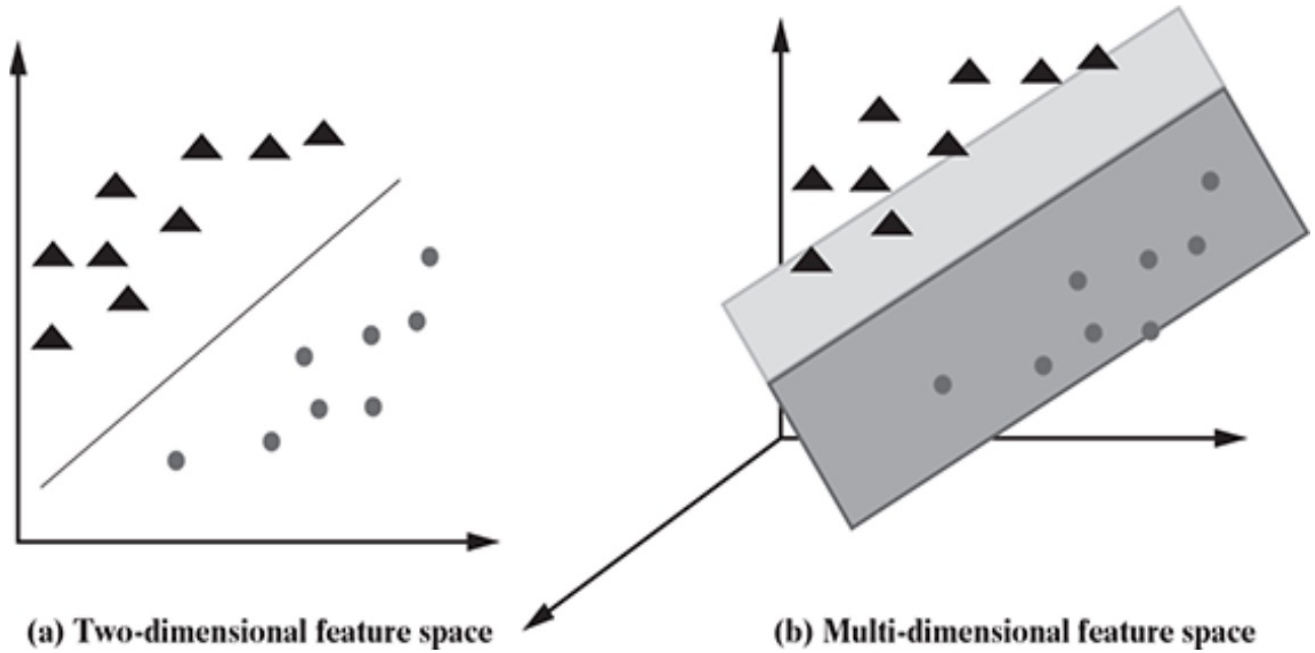
Strengths, Weaknesses, and Applications

- As the forest is built, it generates an internal, unbiased estimate of generalization error, aiding in model evaluation.
- Trained forests can be saved and reused for future predictions on new data.
- Finally, random forest is flexible, capable of solving both classification and regression problems, making it a popular choice for a wide range of machine learning tasks
- However, interpretability is reduced compared to single decision trees, as the ensemble structure is complex.
- Additionally, training multiple trees increases computational cost.
- Despite these drawbacks, random forests are widely used for classification and regression tasks.
- Their versatility and strong performance make them popular among practitioners for applications like fraud detection, medical diagnosis, and remote sensing.

Support Vector Machines (SVM)

- Support Vector Machines (SVMs) are supervised learning models used for both classification and regression tasks, though they are most commonly applied to classification.
- SVMs work by finding a surface, called a hyperplane, that separates data points of different classes in a multi-dimensional feature space.
- The goal is to build an N-dimensional hyperplane that assigns new instances to one of two possible output classes.
- SVMs are especially effective when the data is linearly separable, but they can also handle non-linear cases using advanced techniques.
- The model represents input instances as points in feature space and seeks to maximize the gap (margin) between classes.
- The side of the hyperplane on which a new data point falls determines its predicted class. SVMs are robust to outliers and can generalize well to unseen data.

Support Vector Machines (SVM)



In the overall training process, the SVM algorithm analyses input data and identifies a surface in the multi-dimensional feature space called the hyperplane.

There may be many possible hyperplanes, and one of the challenges with the SVM model is to find the optimal hyperplane.

Hyperplanes, Margins, and Support Vectors

A **hyperplane** is a flat subspace of dimension $(N-1)$ that separates and classifies data in an N-dimensional feature space.

In 2D, it is a line; in 3D, it is a plane. The general equation of a hyperplane in two dimensions is:

$$c_0 + c_1 X_1 + c_2 X_2 = 0$$

For an N-dimensional space, the hyperplane equation becomes:

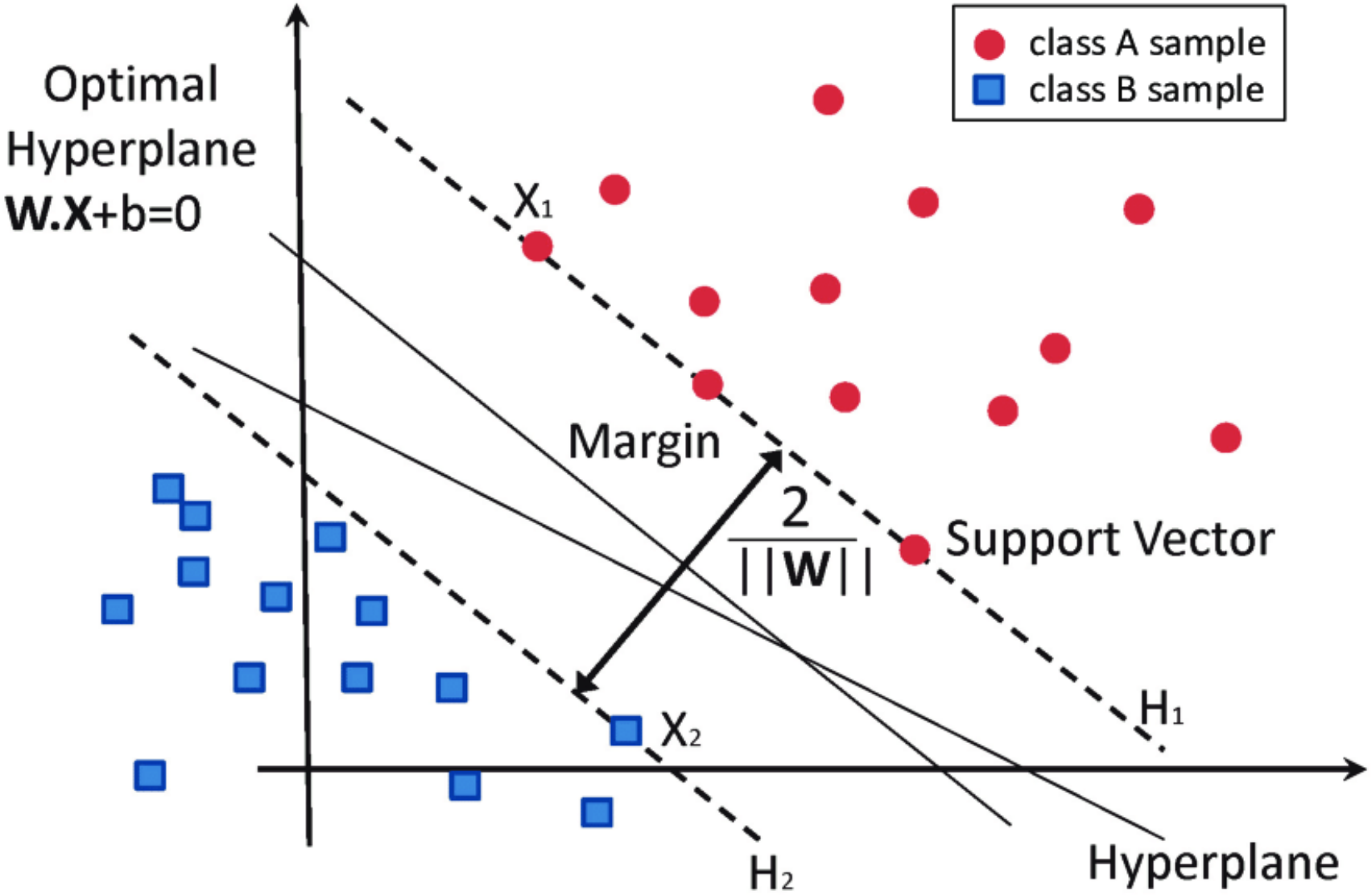
$$c_0 + c_1 X_1 + c_2 X_2 + \ldots + c_N X_N = 0$$

The **margin** is the distance between the hyperplane and the nearest data points from each class, known as the **support vectors**. Maximizing this margin increases the model's confidence and ability to generalize.

Support vectors are the critical data points—removing them would change the hyperplane's position. The Support Vector Machine (SVM) algorithm seeks the *optimal hyperplane* that separates the classes while maximizing the margin.

The further a data point is from the hyperplane, the more confidently it is classified.

Hyperplanes, Margins, and Support Vectors

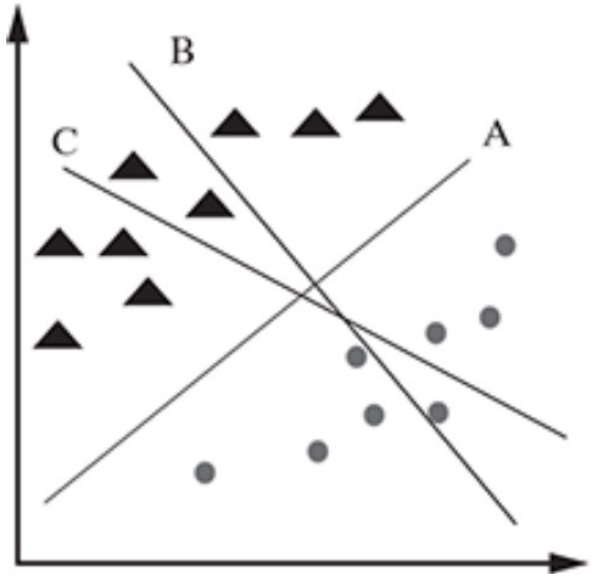


Choosing the Correct Hyperplane

- There can be multiple hyperplanes that separate two classes, but SVM aims to find the one that best segregates the data.
- The correct hyperplane should (1) separate the classes as accurately as possible, (2) maximize the margin between the nearest points of both classes, and (3) prioritize reducing misclassifications over simply maximizing the margin.
- Several scenarios illustrate this: sometimes a hyperplane with a larger margin is preferred, but if it misclassifies points, a smaller-margin hyperplane with perfect classification is chosen.
- SVM is also robust to outliers, as it can ignore them to find the best hyperplane.
- The process involves analyzing the data, identifying possible hyperplanes, and selecting the one that offers the best trade-off between margin and classification accuracy.

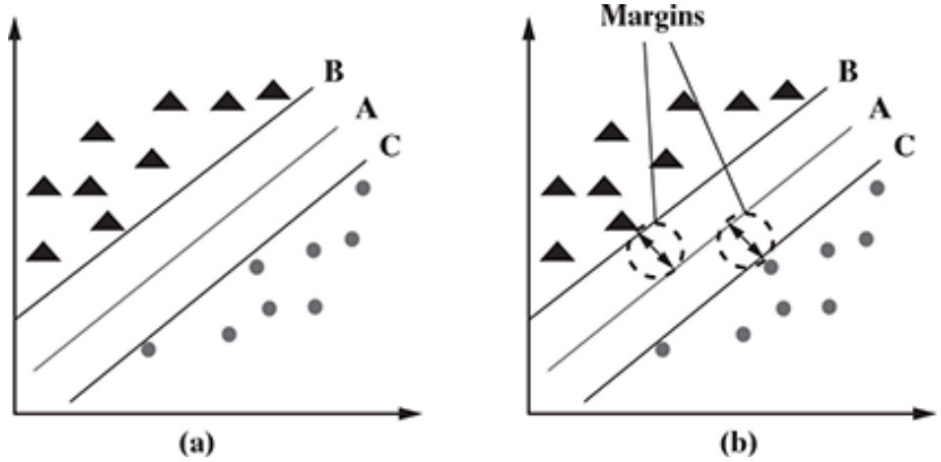
Choosing the Correct Hyperplane

- Scenario 1 : There are multiple possible hyperplanes (A, B, and C) that can separate the two classes (e.g., triangles and circles).
- The correct hyperplane is the one that best segregates the two classes, meaning it separates them with no misclassifications. In this scenario, hyperplane ‘A’ is identified as the best because it perfectly separates the classes.
-



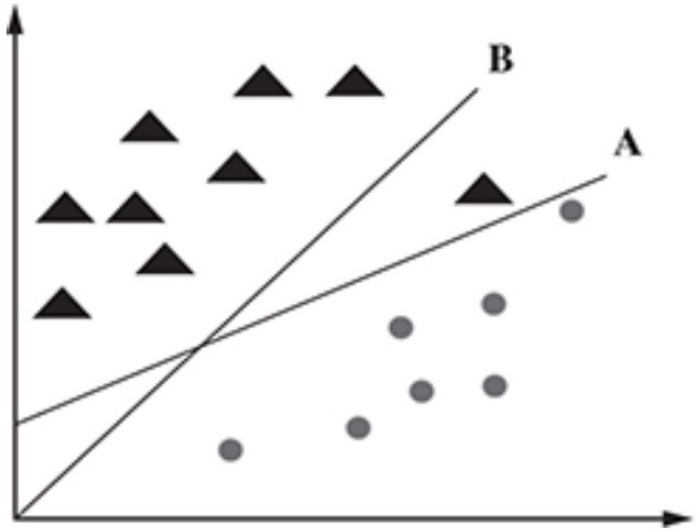
Choosing the Correct Hyperplane

- Scenario 2 : Again, there are several hyperplanes (A, B, and C) that separate the classes.
- The decision is made by maximizing the margin—the distance between the nearest data points of both classes and the hyperplane.
- The hyperplane with the largest margin (here, hyperplane A) is chosen, as it is more robust and less likely to misclassify new data.



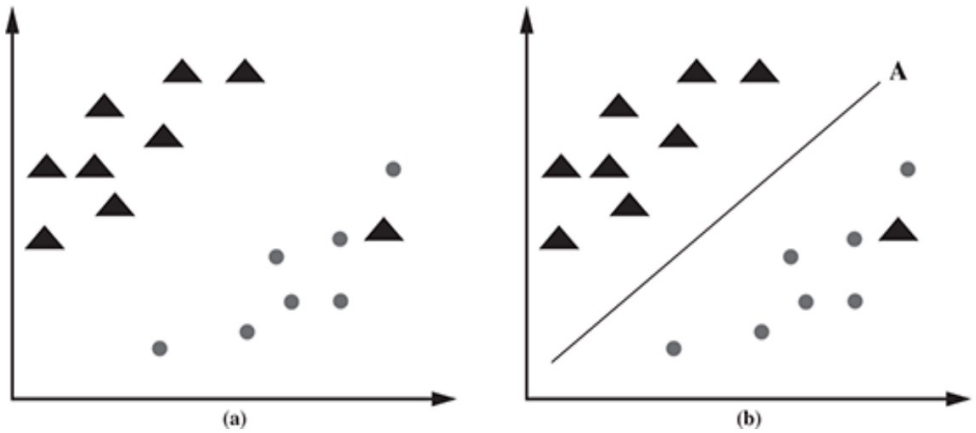
Choosing the Correct Hyperplane

- Scenario 3 :In this scenario, one hyperplane (B) has a larger margin but misclassifies some points, while another (A) classifies all points correctly but with a smaller margin.
- SVM selects the hyperplane which classifies the classes accurately before maximizing the margin.
- Here, hyperplane B has a classification error, and A has classified all data instances correctly.
- The correct hyperplane is the one that minimizes misclassification, so hyperplane A is chosen, even though its margin is smaller.



Choosing the Correct Hyperplane

- Scenario 4 : Here, the data cannot be perfectly separated due to outliers (e.g., a triangle among circles).
- SVM is robust to such outliers and selects the hyperplane (A) that maximizes the margin for the majority of the data, effectively ignoring the outlier.

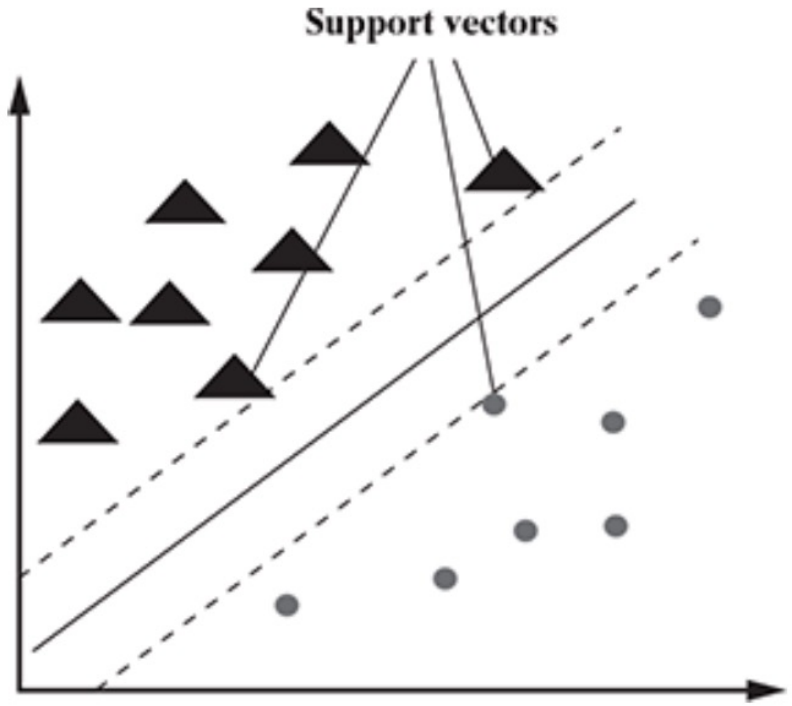


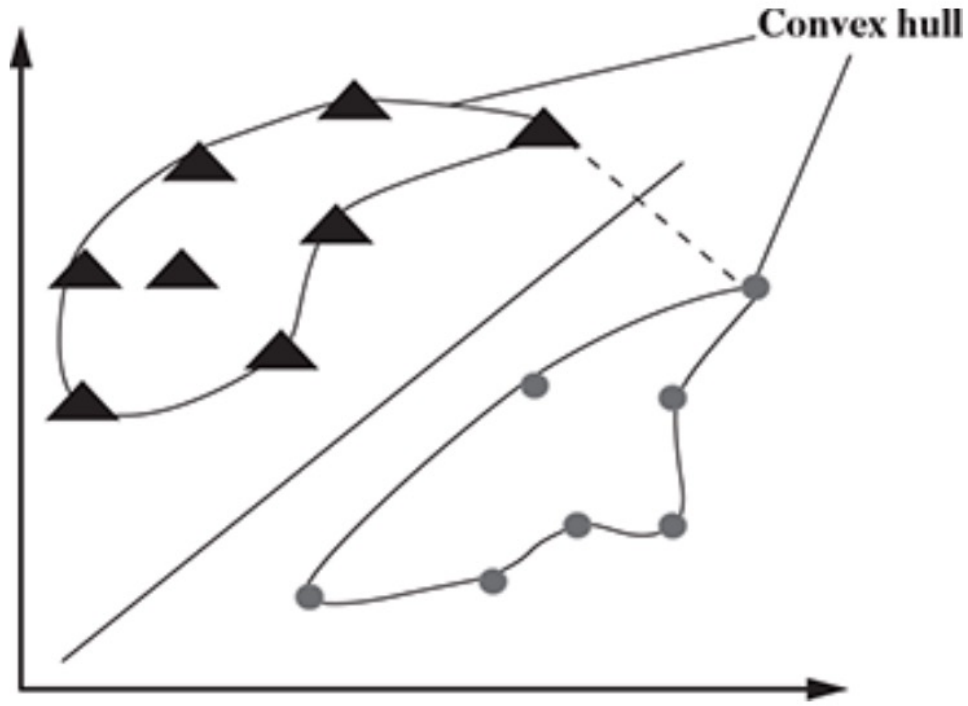
- **These scenarios illustrate that the correct hyperplane should (1) separate the classes as accurately as possible, (2) maximize the margin, and (3) prioritize reducing misclassifications, even in the presence of outliers.**

Maximum Margin Hyperplane (MMH)

- The **Maximum Margin Hyperplane (MMH)** is the hyperplane with the largest possible separation between classes.
- Maximizing the margin leads to better generalization and fewer classification errors on new data.
- For linearly separable data, the MMH can be found by drawing convex hulls around each class and then finding the perpendicular bisector of the shortest line connecting the hulls.
- Mathematically, the SVM optimization problem seeks to minimize $||w||$ (the norm of the weight vector) subject to constraints that ensure correct classification.
- For non-linearly separable data, SVM introduces slack variables (ξ) to allow some misclassifications (soft margin), and a cost parameter C to penalize these errors.
- The optimization then balances maximizing the margin and minimizing misclassification cost.

Maximum Margin Hyperplane (MMH)





the task of SVM is to solve the optimization problem

$$\min\left(\frac{1}{2}\vec{c}^2\right).$$

Maximum Margin Hyperplane (MMH)

- **Hard margin SVM** requires data to be linearly separable and strictly enforces no misclassifications, making it sensitive to outliers.
- **Soft margin SVM** is more flexible, allowing for some misclassifications by introducing a penalty for violations, which makes it suitable for real-world, non-linearly separable, or noisy data.
- In real-world scenarios, data is often not perfectly separable, and some misclassifications or margin violations are inevitable.

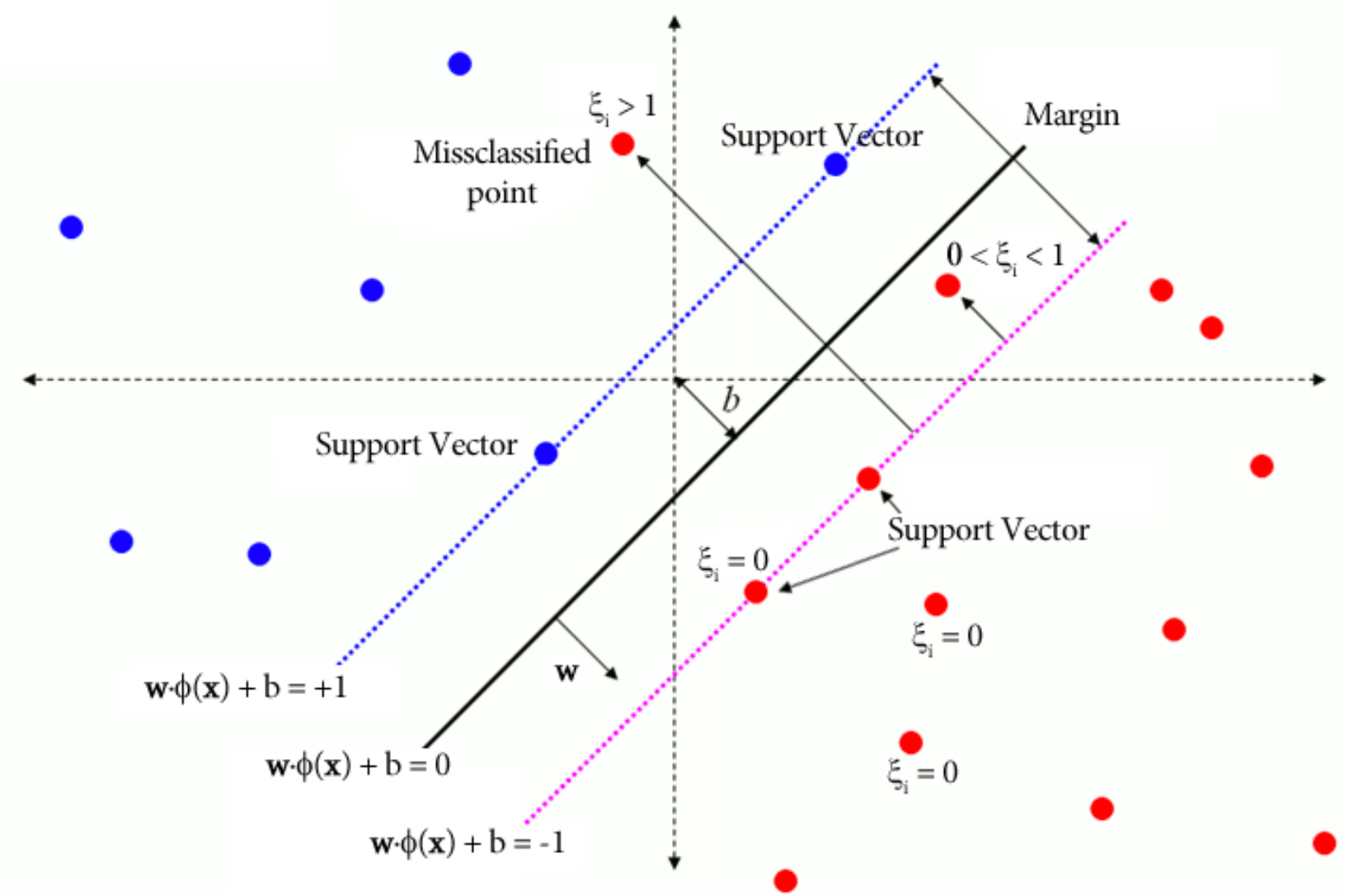
Maximum Margin Hyperplane (MMH)

- **Handling Non-Separable Data**: Slack variables, denoted as (ξ) , allow for some data points to either fall within the margin or even be misclassified on the wrong side of the separating hyperplane.
- Each slack variable corresponds to a specific data point.
- The value of a slack variable ξ quantifies the degree of violation for a given data point x:
- If ($\xi = 0$), the data point is correctly classified and lies outside the margin.
- If ($0 < \xi <= 1$)the data point is correctly classified but lies within the margin.
- If ($\xi > 1$)the data point is misclassified and lies on the wrong side of the hyperplane.

Maximum Margin Hyperplane (MMH)

- **Soft Margin Optimization:**
- In soft margin SVM, the optimization problem is modified to include the slack variables.
- The objective function aims to minimize both the margin violation (represented by the sum of slack variables) and the inverse of the margin width.
- This is often achieved by adding a penalty term.
- The parameter C controls the trade-off between maximizing the margin and minimizing the classification errors.
- A large C value assigns a high penalty to misclassifications.
- A small C value allows for more misclassifications and margin violations

Maximum Margin Hyperplane (MMH)



the task of SVM is to solve the optimization problem

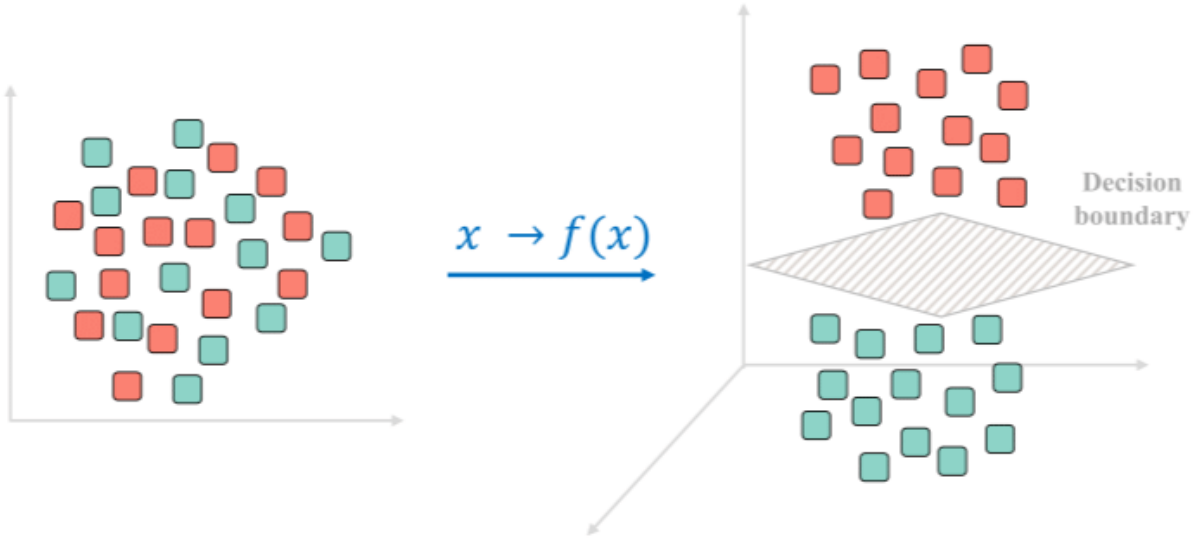
$$\min\left(\frac{1}{2}\vec{c}^2\right) + C\sum_{i=1}^{N}\xi_i$$

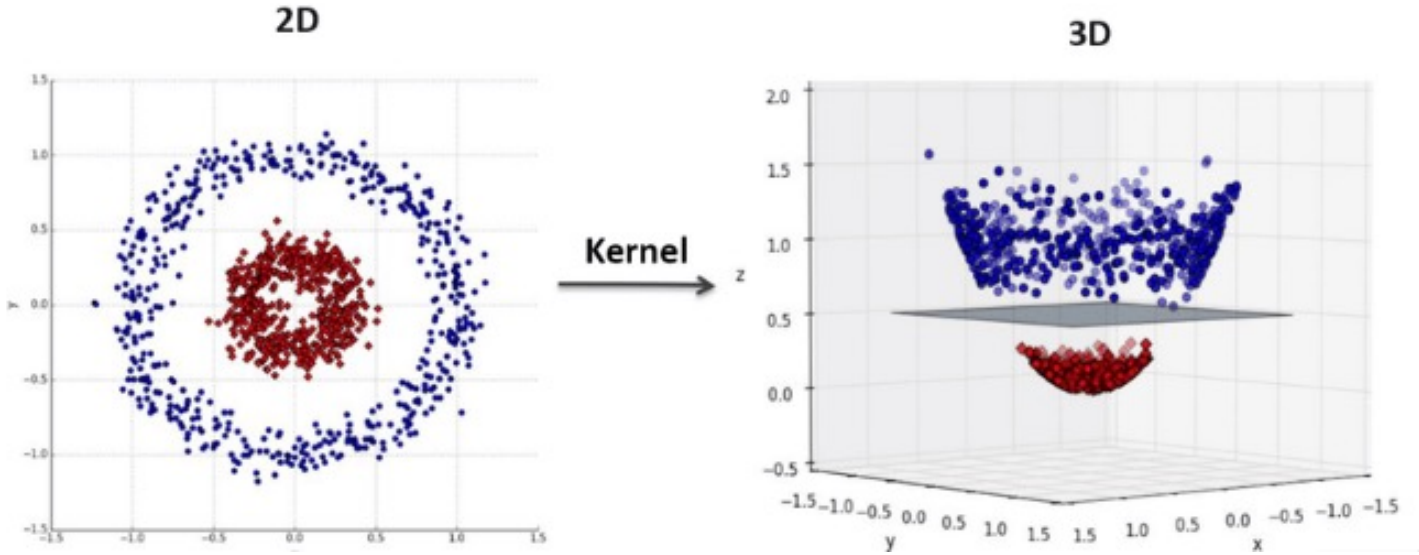
The Kernel Trick for Nonlinear Data

- When data is not linearly separable, SVM uses the kernel trick to transform the input space into a higher-dimensional space where a linear separator may exist.
- Kernels are functions that compute the similarity between data points in this new space without explicitly performing the transformation, making computation efficient.
- Common kernel functions include:
  - **Linear kernel:** $K(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y}$
  - **Polynomial kernel:** $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y} + c)^d$
  - **Sigmoid kernel:** $K(\mathbf{x}, \mathbf{y}) = \tanh(\alpha \mathbf{x} \cdot \mathbf{y} + c)$
  - **Gaussian RBF kernel:** $K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$

The effectiveness of SVM depends on the choice of kernel and its parameters. The kernel trick enables SVM to handle complex, non-linear relationships in data.

The Kernel Trick for Nonlinear Data





Strengths, Weaknesses, and Applications of SVM

Strengths: SVMs can be used for both classification and regression. They are robust to noise and outliers, and often produce highly accurate predictions. SVMs are especially effective for binary classification problems and can handle high-dimensional data well.

Weaknesses: SVMs are mainly suited for binary classification; multi-class problems require additional strategies. The model can be complex and difficult to interpret, especially in high dimensions. SVMs can be slow and memory-intensive for large datasets with many features or instances.

Applications: SVMs are widely used in bioinformatics (e.g., cancer detection), image recognition (e.g., face detection), and other fields requiring binary classification. Their ability to generalize well makes them popular for a variety of machine learning tasks.

Note: Next sections we will look at regression. It must me noted at the end we will see how linear regression builds the foundation for logistic regression.

However Logistic regression is a classifier and is not used for regression tasks.

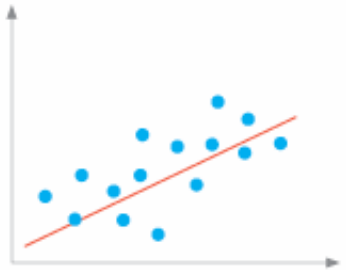
Regression Models

- Regression is a supervised learning method aimed at predicting numerical outcomes (continuous variables) from one or more predictors.
- It models the relationship between a dependent variable *Y* and one or more independent variables *X* through a function $Y=f(X)$.
- Regression models are widely applied for forecasting tasks such as real estate pricing, weather prediction, and demand estimation.
- The key objective is to find the best fit line or curve that minimizes prediction error, typically measured using methods like *Ordinary Least Squares (OLS)*.

Regression Models

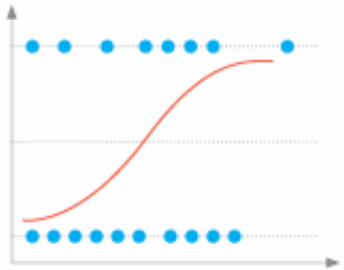
- Common types of regression include simple linear regression (one predictor), multiple linear regression (multiple predictors), polynomial regression (higher-degree terms for non-linear data), and logistic regression (categorical outcomes).
- Assumptions underlying regression include linearity, independence of errors, homoskedasticity (constant variance), and normal distribution of residuals.
- The *Gauss-Markov theorem* states that under these assumptions, OLS provides the *Best Linear Unbiased Estimator (BLUE)*.
- Regression models are highly interpretable, enabling both prediction and inference about the strength and direction of relationships among variables.

5 types of regression



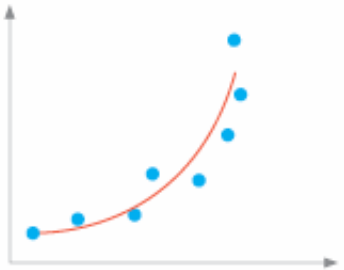
Linear regression

Predicts a continuous output by modeling a straight-line relationship between input features and target variables, such as estimating the impact of price changes on demand.



Logistic regression

Models the probability of binary outcomes, such as predicting customer churn; commonly used in classification tasks.

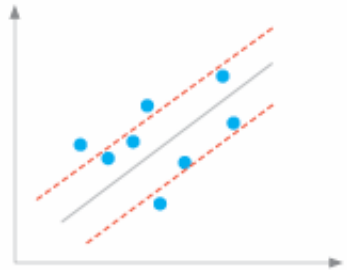


Polynomial regression

Captures nonlinear relationships, such as estimating the impact of ad spending on sales, by fitting a polynomial curve to data points.

Time series regression

Predicts future values in a time-dependent data set; often employed to forecast future values based on past observations, as seen in stock market analysis.



Support vector regression

Approximates a continuous function by identifying a hyperplane that best represents the data's structure; valuable in various applications, including financial market prediction.

The important ones for now..

- **Linear regression.** Linear regression models assume a linear relationship between a target and predictor variables. The model aims to fit a straight line representing the data points. Linear regression is useful when there is a linear relationship between the variables, such as predicting sales based on advertising expenditure or estimating the impact of price changes on demand.
- **Logistic regression.** Logistic regression is used when the target variable is binary or has two classes. It models the probability of an event occurring -- for example, yes/no or success/failure -- based on predictor variables. Logistic regression is commonly used in business contexts for binary classification tasks such as customer churn prediction or transaction fraud detection.
- **Polynomial regression.** Polynomial regression extends linear regression by incorporating polynomial concepts such as quadratic and cubic equations to format the predictor variables and capture cases where a straightforward linear relationship doesn't exist, such as estimating the impact of ad spending on sales

Regression vs Classification

enjoy algorithms

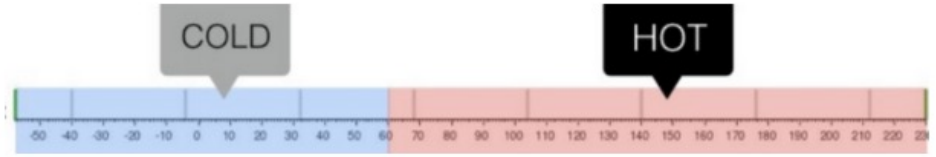
Regression

What will be the temperature tomorrow?

Fahrenheit

Classification

Will it be hot or cold tomorrow?



Fahrenheit

Regression vs Classification

- Formal definition: Regression is a type of problem that uses machine learning algorithms to learn the continuous mapping function.
- Taking the example shown in the above image, we want our machine learning algorithm to predict the weather temperature for today.
- The output would be continuous if we solved the above problem as a regression problem.
- It means our ML model will give exact temperature values, e.g., 24 °C, 24.5°C, 23°C etc.

Simple Linear Regression

Simple Linear Regression is the foundation of regression analysis and involves one predictor and one response variable.

It assumes a linear relationship between X and Y, represented as $Y = a + bX + \varepsilon$, where a is the intercept, b is the slope, and ε is the error term.

The slope represents the average change in Y for a one-unit change in X.

The model parameters (a, b) are estimated using the *rdinary Least Squares (OLS) technique, which minimizes the Sum of Squared Errors (SSE) between predicted and actual values.

Positive slopes indicate a direct relationship, while negative slopes represent an inverse relationship.

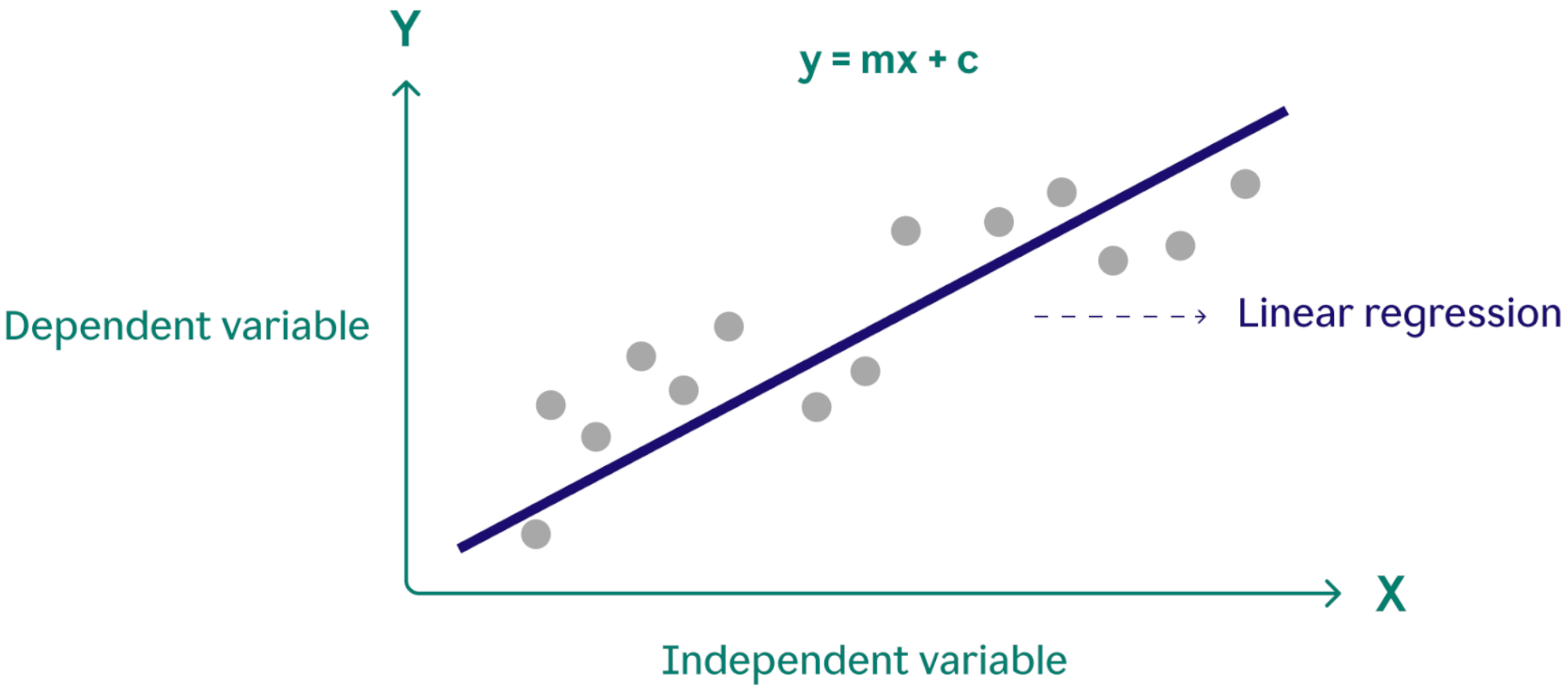
Simple Linear Regression

The regression line provides a best-fit estimate across data points, and the residuals (prediction errors) indicate deviations. For example, predicting exam performance where internal marks (X) influence external marks (Y).

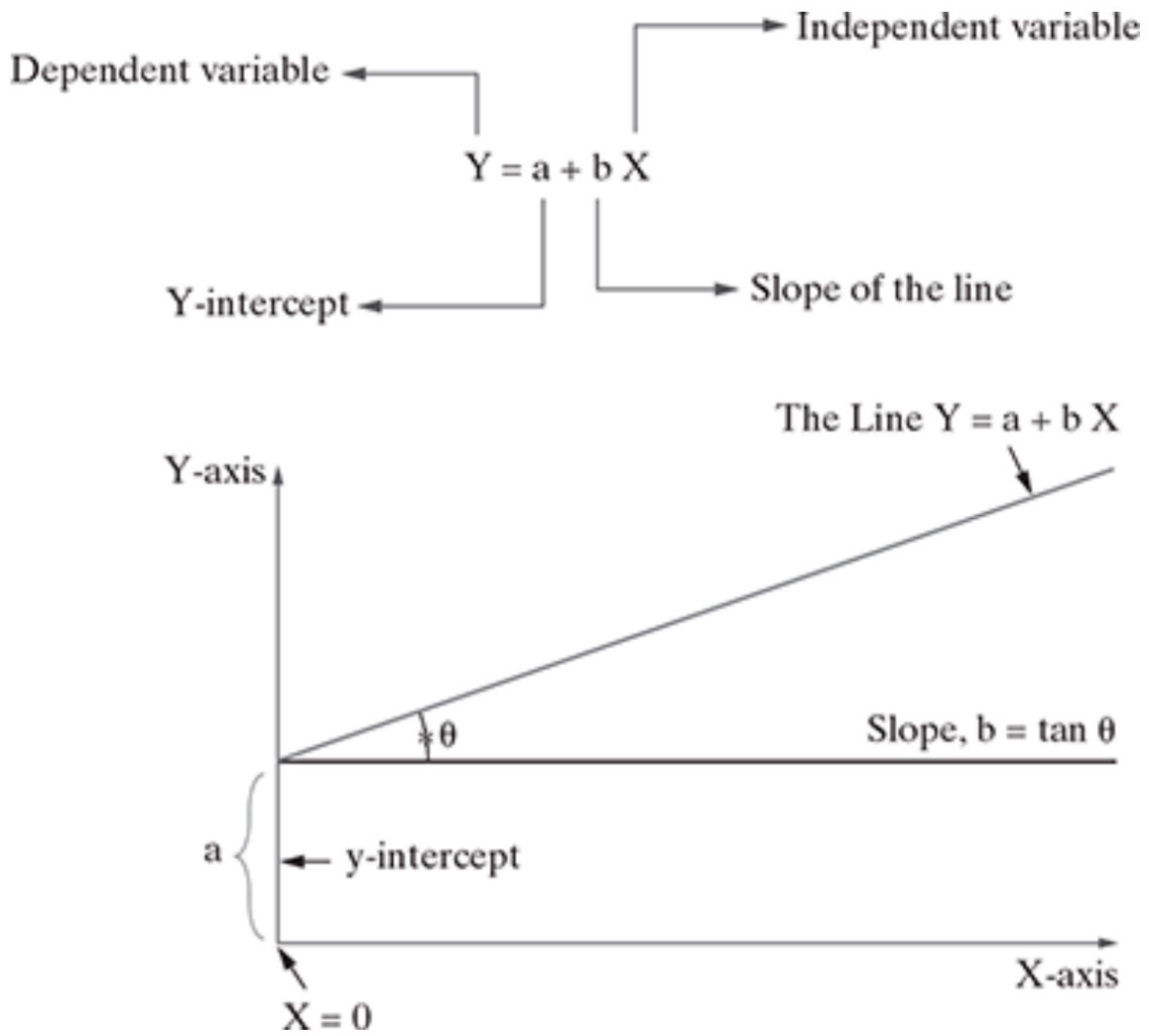
The OLS algorithm includes steps like calculating means, deviations, and slope-intercept estimates.

Graphical analysis through scatter plots helps visualize positive, negative, or no correlation. Simple regression forms the building block for complex regression types, establishing intuition for modeling continuous data relationships.

Simple Linear Regression



Simple Linear Regression

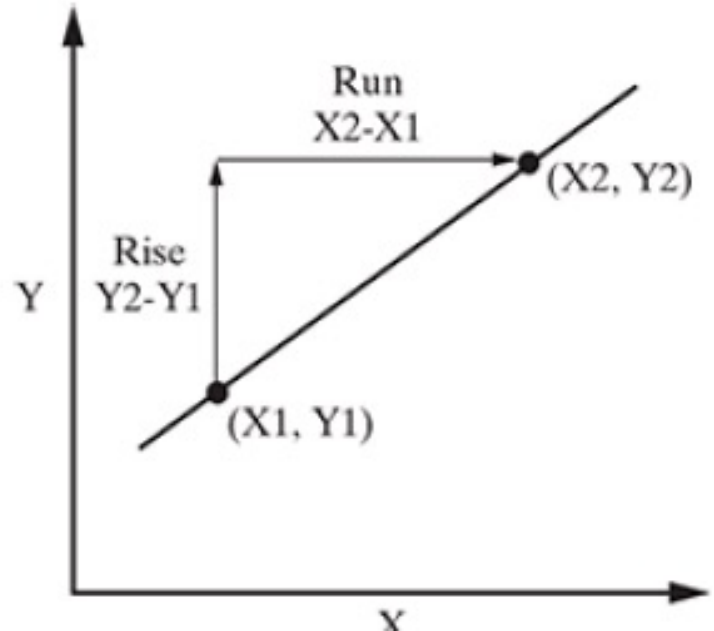


Linear Regression

- When there is a single input variable, the regression is referred to as Simple Linear Regression.
- We use the single variable (independent) to model a linear relationship with the target variable (dependent).
- We do this by fitting a model to describe the relationship.
- If there is more than predicting variable, the regression is referred to as Multiple Linear Regression.

Slope of the simple linear regression model

- Slope of a straight line represents how much the line in a graph changes in the vertical direction (Y-axis) over a change in the horizontal direction (X-axis) as shown
- **Slope = Change in Y / Change in X**
- Rise is the change in Y-axis (Y2− Y1) and Run is the change in X-axis (X2 − X1). So, slope is represented as given below:
- **Slope = Rise/Run = (Y2 — Y1) / (X2 — X1)**



Slope of the simple linear regression model

Types of Slopes

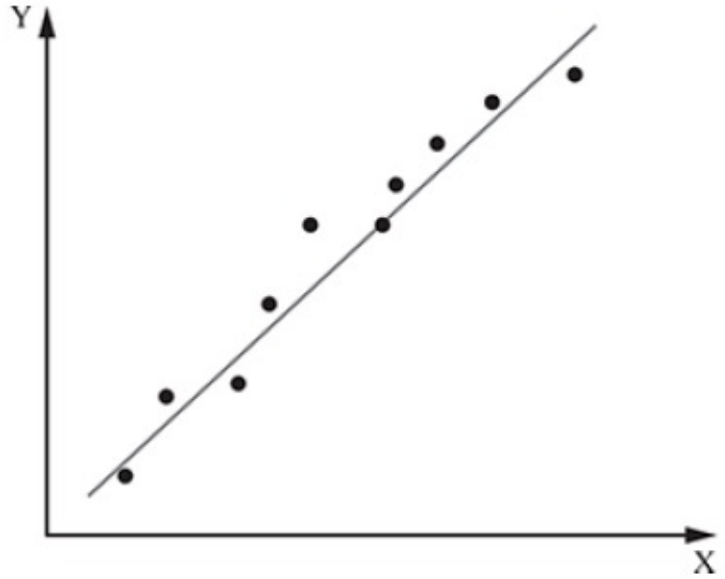
- There can be two types of slopes in a linear regression model: positive slope and negative slope.
- Different types of regression lines based on the type of slope include
  - Linear positive slope
  - Curve linear positive slope
  - Linear negative slope
  - Curve linear negative slope

Linear positive slope

- A positive slope always moves upward on a graph from left to right
- A positive slope indicates a direct relationship between two variables, typically denoted as X and Y.
- As the value of the independent variable (X) increases, the value of the dependent variable (Y) also increases.
- The slope is a measure of the steepness of the line and is calculated as the ratio of the change in Y (Delta (Y)) to the change in X (Delta(X))
- Slope = Rise/Run = (Y2 − Y1) / (X2 − X1) = Delta (Y) / Delta(X)

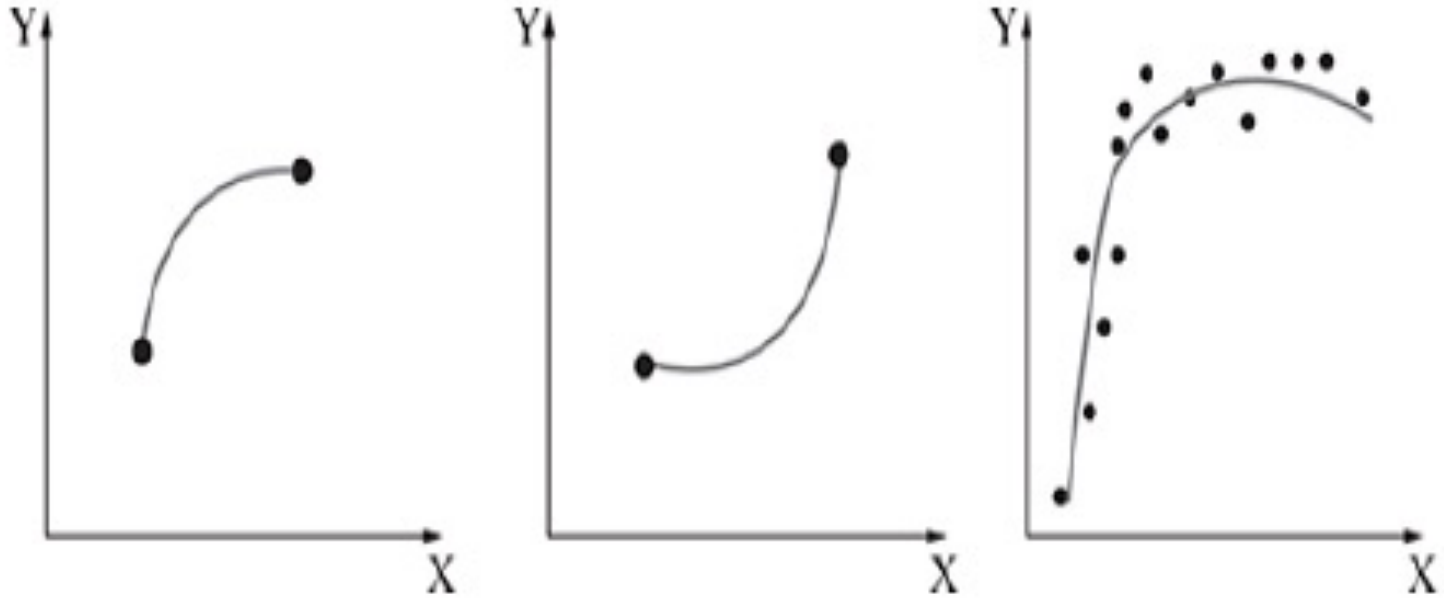
Linear positive slope

- **Scenario 1 for positive slope:** Delta (Y) is positive and Delta (X) is positive
- **Scenario 2 for positive slope:** Delta (Y) is negative and Delta (X) is negative



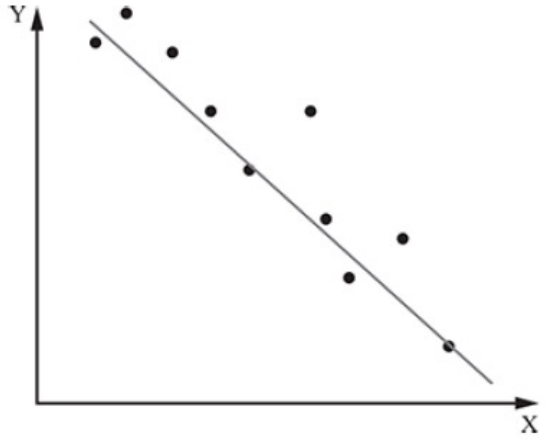
Curve linear positive slope

- Curves in these graphs slope upward from left to right.
- Slope for a variable (X) may vary between two graphs, but it will always be positive; hence, the graphs are called as graphs with curve linear positive slope



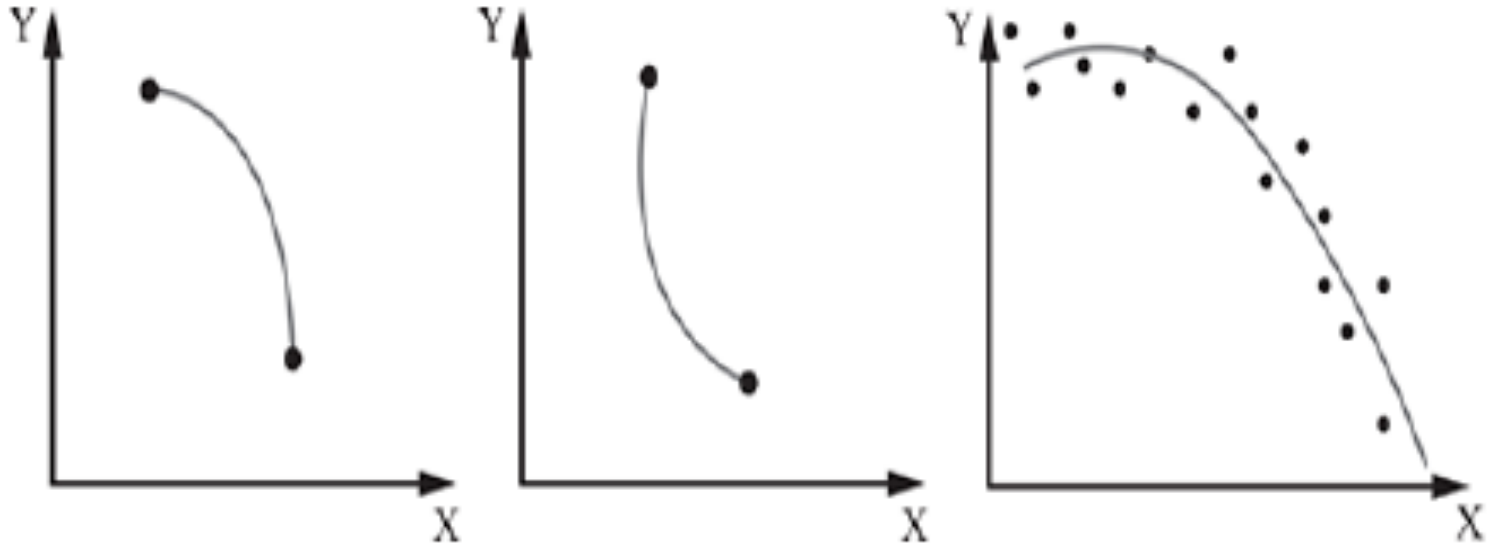
Linear negative slope

- A negative slope always moves downward on a graph from left to right.
- As X value (on X-axis) increases, Y value decreases
- **Scenario 1 for negative slope:** Delta (Y) is positive and Delta (X) is negative
- **Scenario 2 for negative slope:** Delta (Y) is negative and Delta (X) is positive



Curve linear negative slope

- Curves in these graphs slope downward from left to right.
- Slope for a variable (X) may vary between two graphs, but it will always be negative; hence, the graphs are called as graphs with curve linear negative slope.



Error in simple regression

In simple linear regression, error refers to the difference between the observed value and the value predicted by the regression line.

This difference is called a residual, denoted as $e_i = y_i - \hat{y}_i$, where y_i is the actual value and $\hat{y}_i$ is the predicted value.

The collection of all residuals helps assess the model's accuracy.

The Sum of Squared Errors (SSE) is a key metric, calculated as $SSE = \sum(y_i - \hat{y}_i)^2$, and is minimized during model fitting.

Ordinary Least Squares (OLS)

The **Ordinary Least Squares (OLS)** algorithm is the standard method for fitting a regression line to data. OLS finds the line that minimizes the sum of squared residuals (errors) between observed and predicted values. The regression equation is $y = b_0 + b_1 x$, where b_0 is the intercept and b_1 is the slope. OLS estimates these parameters using formulas:

$$b_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

$$b_0 = \bar{y} - b_1 \bar{x}$$

Here, $\bar{x}$ and $\bar{y}$ are the means of the independent and dependent variables, respectively. OLS assumes linearity, constant error variance, and independent errors. When these conditions are met, OLS provides the *Best Linear Unbiased Estimator (BLUE)*. The algorithm is widely used due to its simplicity and interpretability.

OLS algorithm

Step 1: Calculate the mean of X and Y

Step 2: Calculate the errors of X and Y

Step 3: Get the product

Step 4: Get the summation of the products

Step 5: Square the difference of X

Step 6: Get the sum of the squared difference

Step 7: Divide output of step 4 by output of step 6 to calculate ‘b’

Step 8: Calculate ‘a’ using the value of ‘b’

OLS algorithm

Step 2 Step 3 Step 5

X	Y	X- mean (X)	Y- Mean (Y)	$(Xi-\overline{X})(Yi-\overline{Y})$	$(Xi-\overline{X})^2$
15	49	-4.93	-7.8	38.454	24.3049
23	63	3.07	6.2	19.034	9.4249
18	58	-1.93	1.2	-2.316	3.7249
23	60	3.07	3.2	9.824	9.4249
24	58	4.07	1.2	4.884	16.5649
22	61	2.07	4.2	8.694	4.2849
22	60	2.07	3.2	6.624	4.2849
19	63	-0.93	6.2	-5.766	0.8649
19	60	-0.93	3.2	-2.976	0.8649
16	52	-3.93	-4.8	18.864	15.4449
24	62	4.07	5.2	21.164	16.5649
11	30	-8.93	-26.8	239.324	79.7449
24	59	4.07	2.2	8.954	16.5649
16	49	-3.93	-7.8	30.654	15.4449
23	68	3.07	11.2	34.384	9.4249
19.9	**56.8**		$\Sigma(Xi-\overline{X})(Yi-\overline{Y})$	**429.8**	**226.9335**

Step 1 Step 4 Step 6

Step 7: Divide (step4 / step6)

b = 429.28 / 226.93 = 1.89

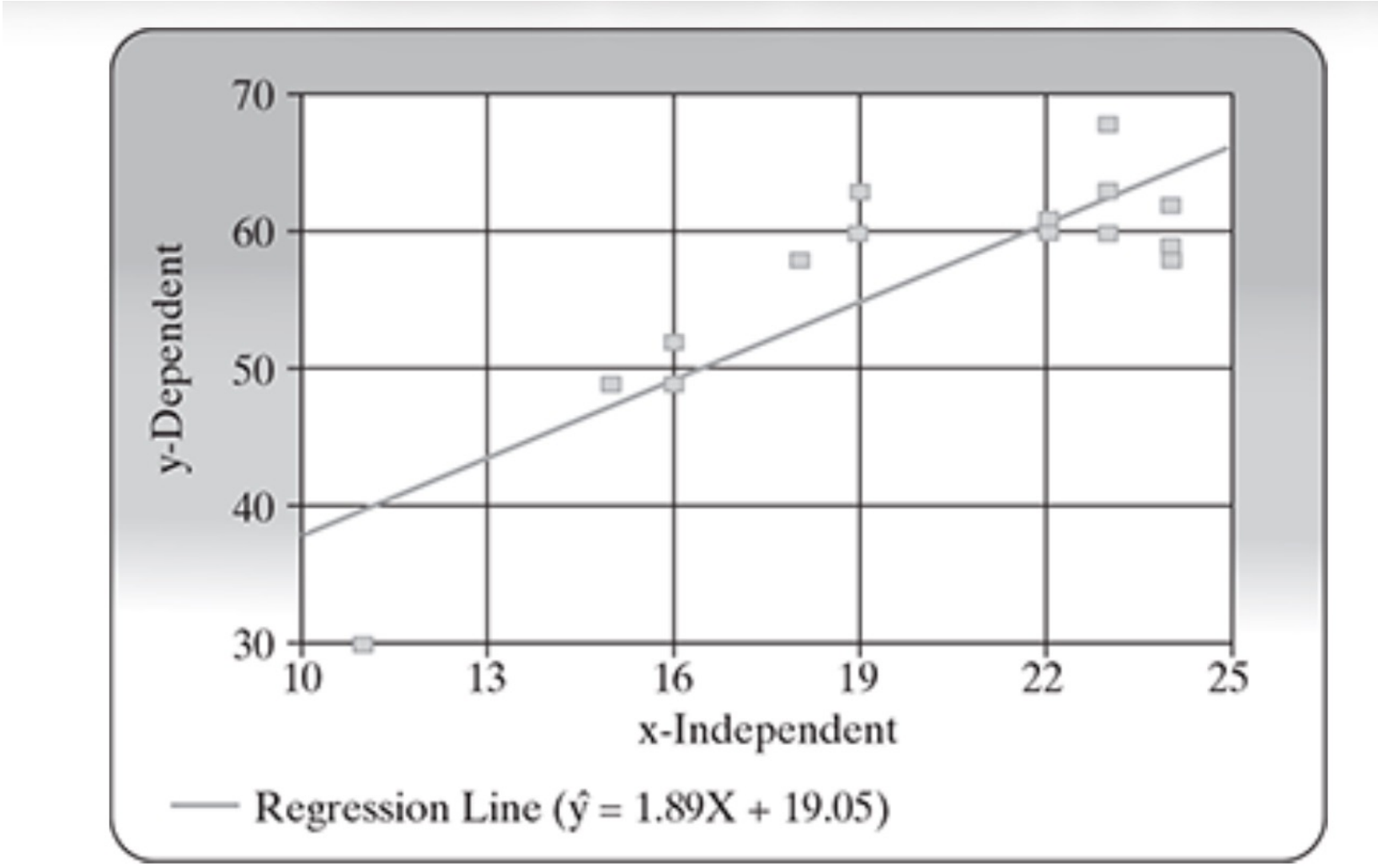
Step 8: Calculate a using the value of b

$a = \overline{Y} - b\overline{X}$

a = 56.8 – 1.89 × 19.9

a = 19.05

OLS algorithm



Assumptions of OLS Regression

- The regression model is linear in parameters (coefficients appear linearly in the equation).
- The mean of the error term (residuals) is zero for any value of the independent variables.
- The variance of the error term is constant across all observations (homoskedasticity).
- The error terms are uncorrelated with each other (no autocorrelation).
- The error term is uncorrelated with the independent variables (no endogeneity).
- No perfect multicollinearity among independent variables (no exact linear relationship between predictors).
- The number of observations is greater than the number of parameters to be estimated.
- The independent variables (regressors) have variability (not all values are the same).
- For inference: The error terms are normally distributed (important for hypothesis testing and confidence intervals).

OLS Assumptions in Simple English

- The relationship between variables is a straight line (linear).
- The average error (difference between actual and predicted values) is zero.
- The errors (residuals) are spread out evenly, not getting bigger or smaller for different values.
- The errors are not related to each other; each error is random.
- The errors are not related to the input variables.
- The input variables are not exact copies of each other.
- There are more data points than things you want to estimate.
- The input values are different from each other (not all the same).
- For testing and confidence intervals, the errors should look like a bell curve (normal distribution).

Multiple Linear Regression

Multiple Linear Regression (MLR) extends the simple regression concept by modeling multiple predictors affecting a single dependent variable simultaneously.

The general equation is $Y = a + b_1X_1 + b_2X_2 + \cdots + b_nX_n + \varepsilon$, where b_i are partial regression coefficients showing each predictor's independent contribution.

MLR assumes a linear, additive relationship among variables, providing a multidimensional plane of best fit.

For instance, predicting property prices (Y) using predictors like area, location, floor, and amenities.

Multiple Linear Regression

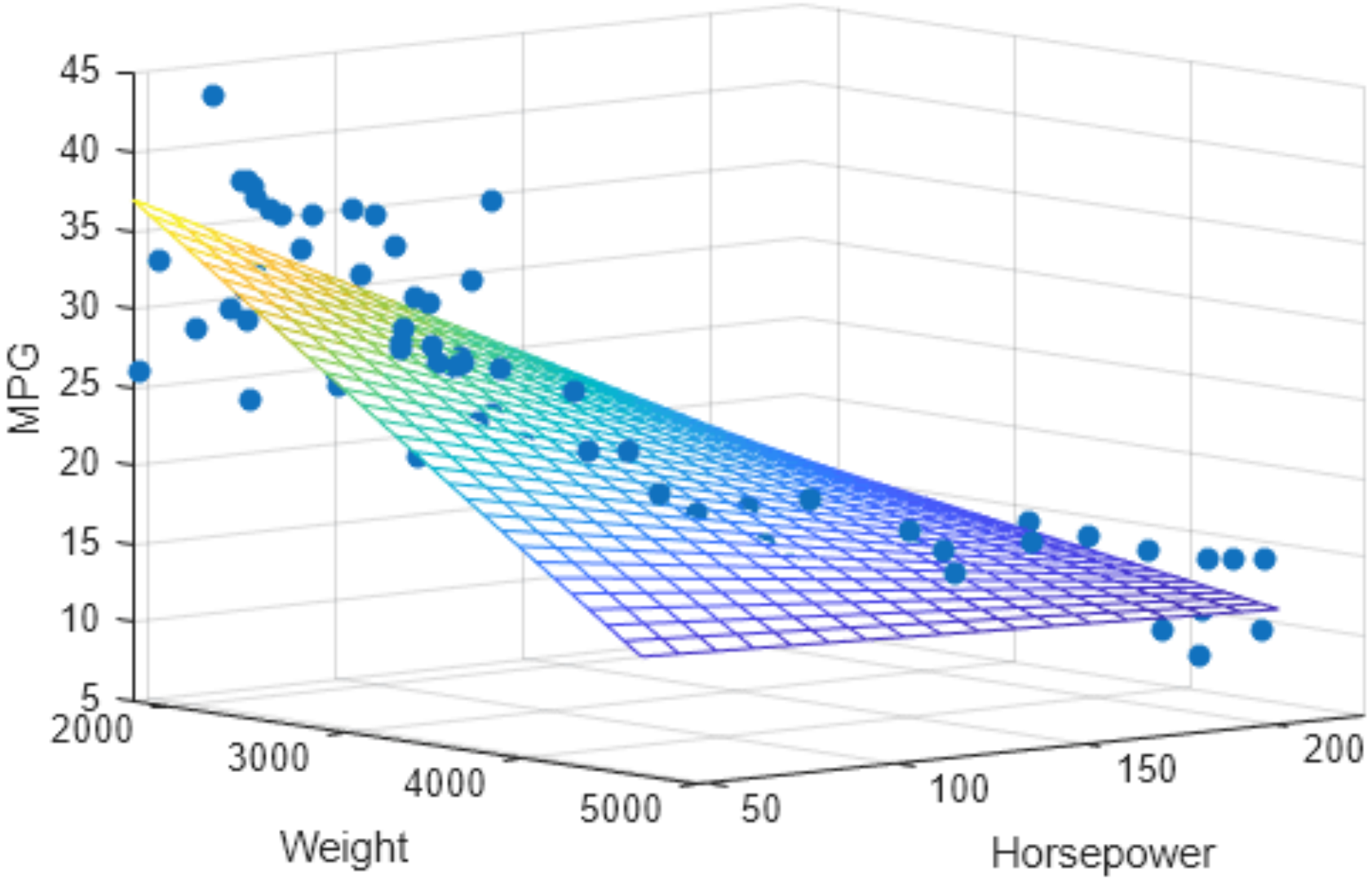
The *OLS* method still applies but must address complexities such as **multicollinearity** (strong intercorrelation among predictors) and **heteroskedasticity** (unequal error variance).

When predictor data are highly correlated or noisy, advanced techniques improve model reliability—such as Ridge (L2) and Lasso (L1) regularization for coefficient shrinkage, subset selection for variable reduction, and dimension reduction (e.g., Principal Component Analysis).

To ensure unbiasedness and accuracy, regression validity must hold assumptions: $n > k$ [n is samples and k is features], zero-mean error, no perfect collinearity, and homoskedastic residuals.

MLR is crucial in multivariable forecasting and inference, balancing interpretability with prediction strength.

Multiple Linear Regression



Main Problems in Regression Analysis

- Multicollinearity: When predictor variables are highly correlated, it becomes hard for the model to separate their individual effects. This can make coefficient estimates unstable and unreliable.
- Overfitting: Adding too many variables or fitting the model too closely to the training data can cause it to capture random noise instead of actual patterns. This usually leads to poor predictions on new data.
- Heteroskedasticity: If the spread of errors changes across values of predictors, OLS assumptions are violated, making predictions less trustworthy.

Main Problems in Regression Analysis

- Autocorrelation: If error terms are related across observations (common in time series), standard regression inferences may break down.
- Sensitivity to Outliers: Outliers can strongly influence the regression line, distorting results. This can make predictions and coefficients misleading.
- Data issues: Missing values, inaccurate data, or low variability in predictors all lower model reliability.
- Incorrect assumptions: If basic OLS requirements (like linearity, normal errors, and constant variance) are not met, regression results may be biased or invalid.

Improving Accuracy of the Linear Regression Model

- Check Assumptions: Always review OLS conditions—linearity, normality, homoskedasticity, no autocorrelation—to be sure model results can be trusted.
- Variable Selection: Use only the most important predictors. Try forward selection, backward elimination, or stepwise approaches to choose significant variables and reduce noise.
- Handle Outliers: Identify outliers (e.g., more than 3 standard deviations from the mean) and remove or correct them to avoid distorting the regression line.
- Address Multicollinearity: Detect correlated predictors using correlation matrices; drop or combine highly similar variables.

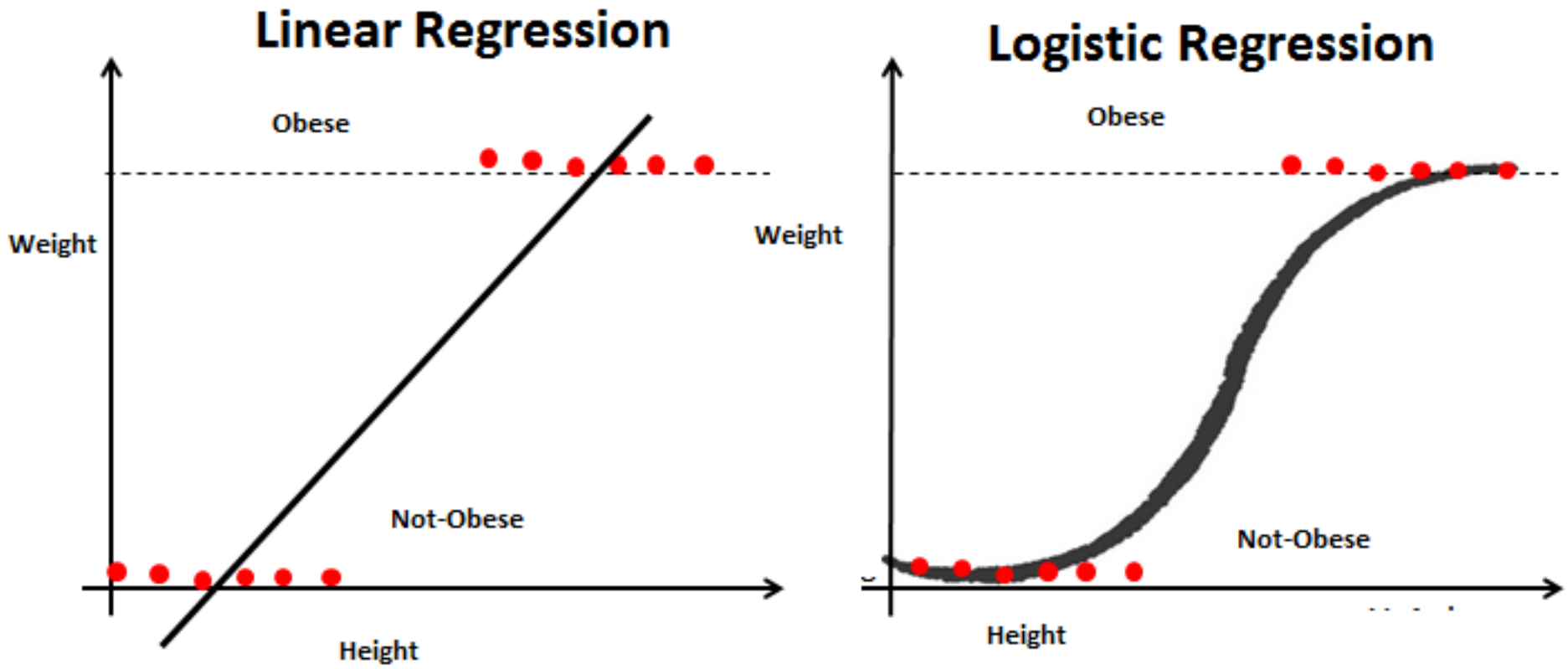
Improving Accuracy of the Linear Regression Model

- Transform Variables: If variables aren’t normally distributed, apply transformations (e.g., log, square root) to achieve better fitting and easier interpretation.
- Impute or Handle Missing Data: Use methods like mean imputation, or drop rows with missing values sparingly to maintain data integrity.
- Evaluate Model Fit: Check R-squared, mean squared error, residual plots, and other metrics to measure predictive quality.
- Cross-validation: Test your model on new (validation or test) data to ensure it generalizes beyond the original dataset.

Logistic Regression

- Linear regression and logistic regression are both popular, easily interpretable methods in supervised machine learning, but they are used for different types of prediction tasks.
- **Linear Regression** : Used to predict a continuous (numeric) outcome, such as price, age, or temperature. It models the relationship between input variable(s) *X* and output *Y* as a straight line and Assumes the change in inputs causes a direct, proportionate change in output. Fitted by minimizing the sum of squared errors (ordinary least squares).
- Example: Predicting house prices based on area, number of rooms, and location.
- **Logistic Regression** : Used to predict a categorical (often binary) outcome, such as yes/no, healthy/sick, or pass/fail. It estimates the probability that the target belongs to a particular category (commonly 0 or 1), using an S-shaped curve and Output values are always between 0 and 1, interpreted as probabilities. Model fitted using maximum likelihood estimation.
- Example: Predicting whether a student will pass or fail based on study hours and attendance.

Logistic Regression



Logistic Regression

Logistic Regression, while named "regression," is primarily a classification algorithm that predicts the probability of a categorical outcome. It models a binary dependent variable ($Y \in \{0, 1\}$) using one or more predictors. The logistic model transforms the linear relationship $a + bX$ using the *sigmoid* (logistic) function:

$$P(Y = 1) = \frac{e^{a+bX}}{1 + e^{a+bX}}$$

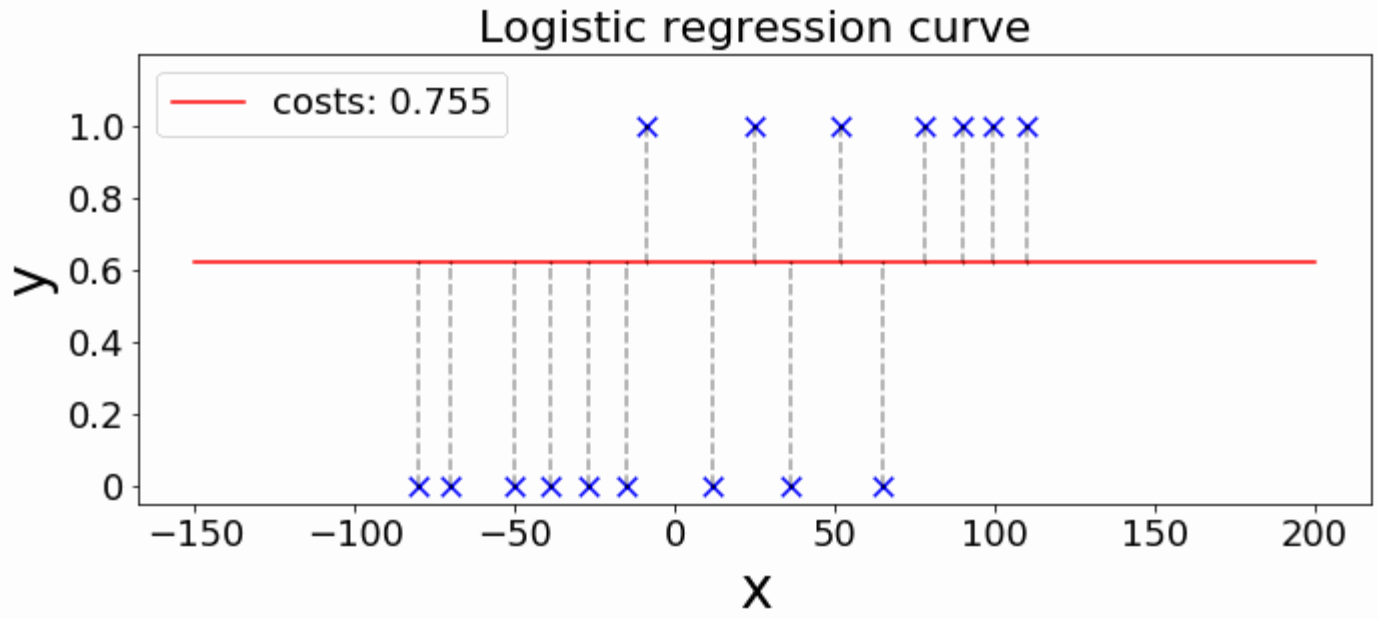
This ensures probabilities remain within $[0, 1]$. Logistic regression estimates coefficients using *Maximum Likelihood Estimation (MLE)*, not OLS. It is often applied in risk prediction (e.g., success/failure, pass/fail, churn/no churn) where relationships are nonlinear on probability scale but linear in log-odds ($\log(\frac{P}{1-P}) = a + bX$).

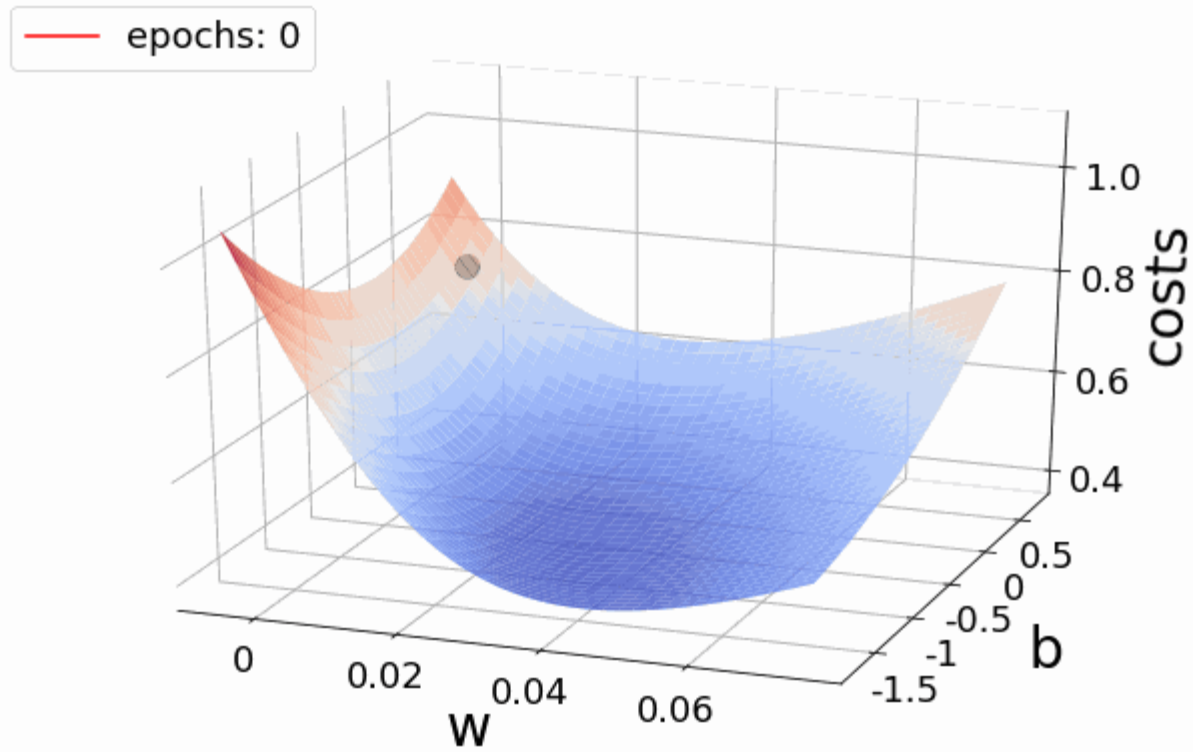
Logistic Regression

Important assumptions include: binary outcome variable, independent and identically distributed (iid) errors, linearity between predictors and the logit function, and no multicollinearity among predictors.

Evaluation relies on chi-square and likelihood ratio tests rather than R^2.

Logistic regression combines interpretability with probabilistic reasoning, providing insights into how input features influence event likelihood while maintaining computational simplicity.


Logistic regression curve
costs: 0.755
1.0
0.8
0.6
0.4
0.2
0
y
−150
−100
−50
0
50
100
150
200
x

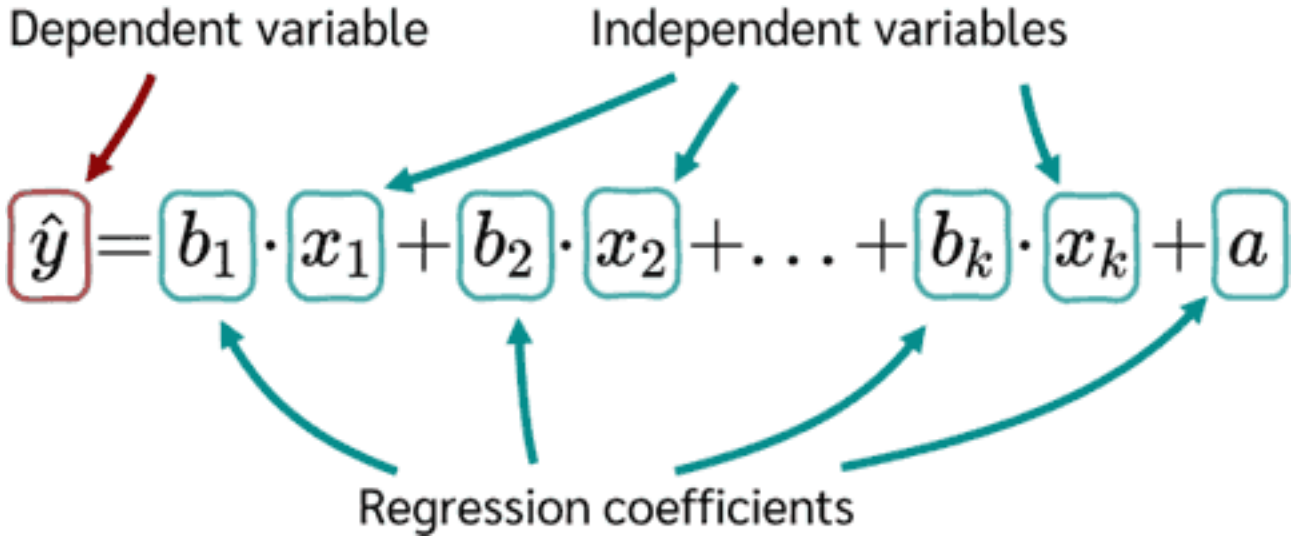

epochs: 0
1.0
0.8
0.6
0.4
costs
0
0.02
0.04
0.06
w
0.5
0
-0.5
-1
-1.5
b

Logistic regression and probabilities

- In linear regression, the independent variables (e.g., age and gender) are used to estimate the specific value of the dependent variable (e.g., body weight).
- In logistic regression, on the other hand, the dependent variable is dichotomous (*0* or *1*) and the probability that expression *1* occurs is estimated. Returning to the example above, this means: How likely is it that the disease is present if the person under consideration has a certain age, sex and smoking status.

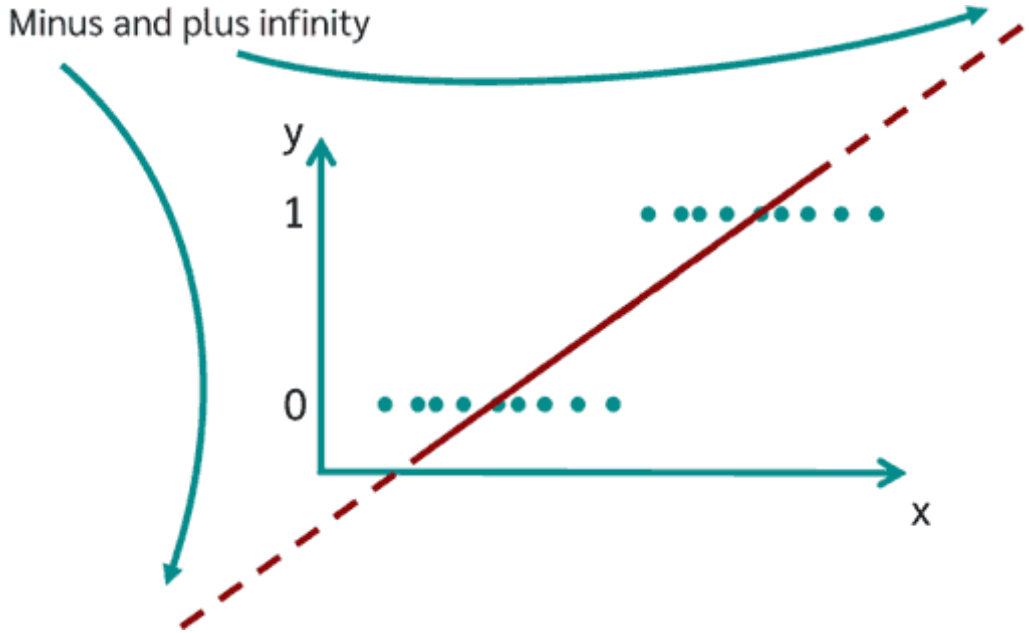
Calculate logistic regression

- To build a logistic regression model, the linear regression equation is used as the starting point.



- However, if a linear regression were simply calculated for solving a logistic regression, the following result would appear graphically:

Calculate logistic regression



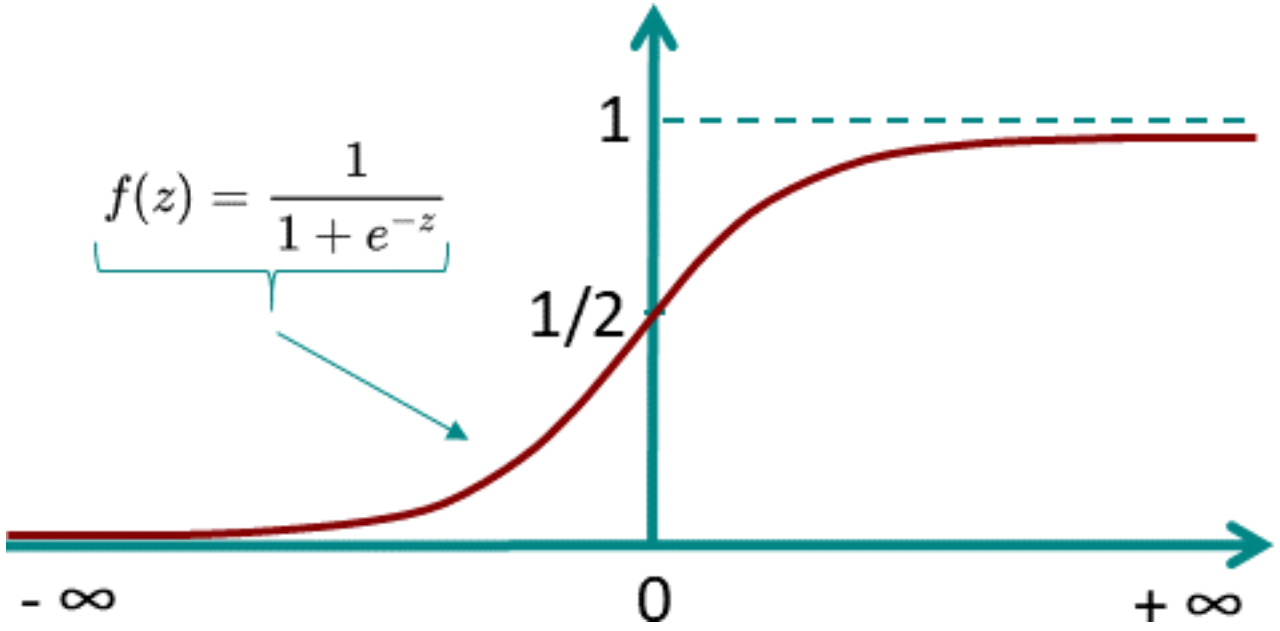
As can be seen in the graph, however, **values between plus and minus infinity** can now occur.

The goal of logistic regression, however, is to estimate the probability of occurrence and not the value of the variable itself. Therefore, the equation must still be transformed.

To do this, it is necessary to restrict the value range for the prediction to the range between 0 and 1. To ensure that only values between 0 and 1 are possible, the **logistic function *f*** is used.

Logistic function

- The logistic model is based on the logical function. The special thing about the logistic function is that for values between minus and plus infinity, it always assumes only values between 0 and 1.



Logistic function

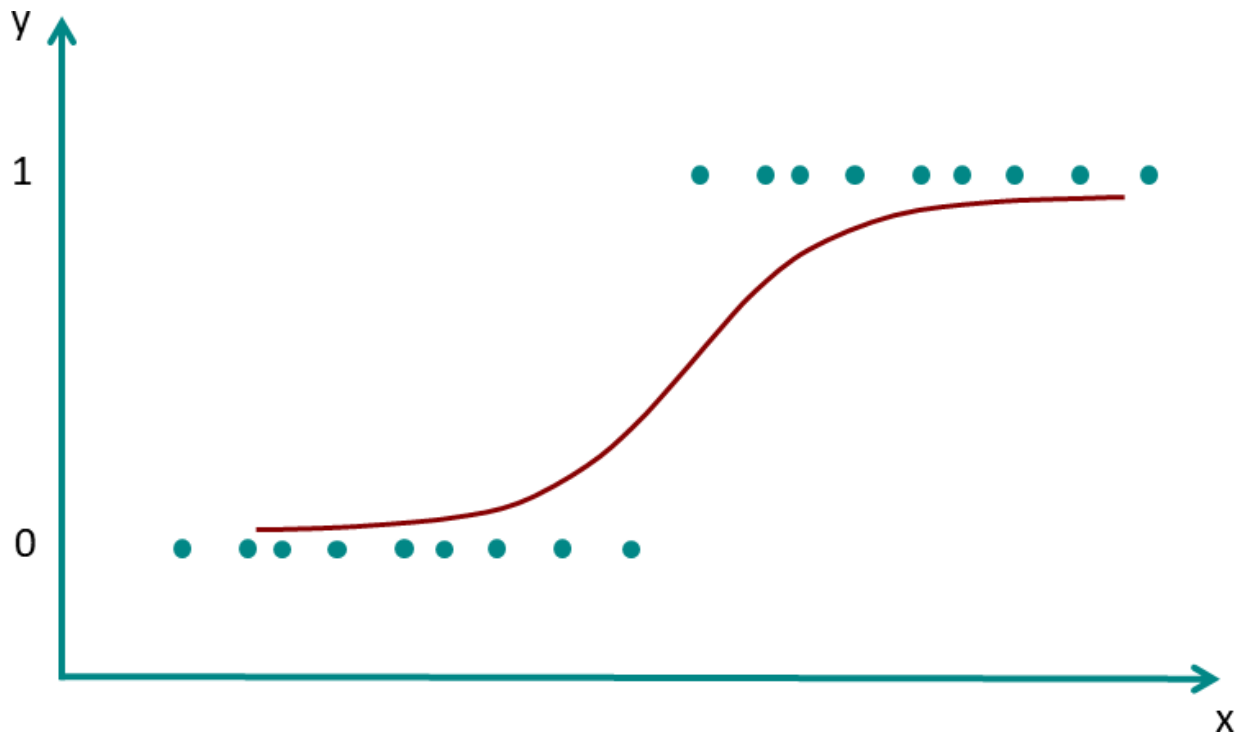
- So the logistic function is perfect to describe the **probability *P(y=1)***.
- If the logistic function is now applied to the regression equation from before the result is:

$$\hat{y} = b_1 \cdot x_1 + b_2 \cdot x_2 + \ldots + b_k \cdot x_k + a$$

$$f(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(b_1 \cdot x_1 + \ldots + b_k \cdot x_k + a)}}$$

Logistic function

- This now ensures that no matter in which range the *x* values are located, only values between 0 and 1 will come out. The new graph now looks like this:



To formalize what we have seen..

The probability that for given values of the independent variable the dichotomous dependent variable *y* is *0* or *1* is given by:

$$P(y=1|x_1,\ldots,x_n) = \frac{1}{1+e^{-(b_1\cdot x_1+\ldots+b_k\cdot x_k+a)}}$$

$$P(y=0|x_1,\ldots,x_n) = 1 - \frac{1}{1+e^{-(b_1\cdot x_1+\ldots+b_k\cdot x_k+a)}}$$

To calculate the probability of a person being sick or not using the logistic regression for the example above, the model parameters b_1, b_2, b_3 and a must first be determined. Once these have been determined, the equation for the example above is:

$$P(\text{Diseased}) = \frac{1}{1+e^{-(b_1\text{Age}+b_2\text{Gender}+b_3\text{Smoking status}+a)}}$$

Maximum Likelihood Method

- To determine the model parameters for the logistic regression equation, the Maximum Likelihood Method is applied.
- The maximum likelihood method is one of several methods used in statistics to estimate the parameters of a mathematical model.
- Another well-known estimator is the least squares method, which is used in linear regression.
- In Maximum Likelihood Method we want to maximize the Likelihood Function.

To understand the **maximum likelihood method**, we introduce the **likelihood function** *L*. *L* is a function of the unknown parameters in the model, in case of logistic regression these are b_1,... b_n, *a*. Therefore we can also write $L(b_1,... b_n, a)$ or $L(\theta)$ if the parameters are summarized in θ.

$L(\theta)$ now indicates how probable it is that the observed data occur. With the change of θ, the probability that the data will occur as observed changes.